

COMP 424 - Artificial Intelligence

Fall 2026

Lecture 3: September 9th, 2026
Informed Search

Su Lin Blodgett (sulin.blodgett@mcgill.ca)
Jackie CK Cheung (jackie.cheung@mcgill.ca)

Uninformed Searches

- Breadth-First Search
- Depth-First Search
- Uniform Cost Search
- Iterative Deepening Search

Key Properties

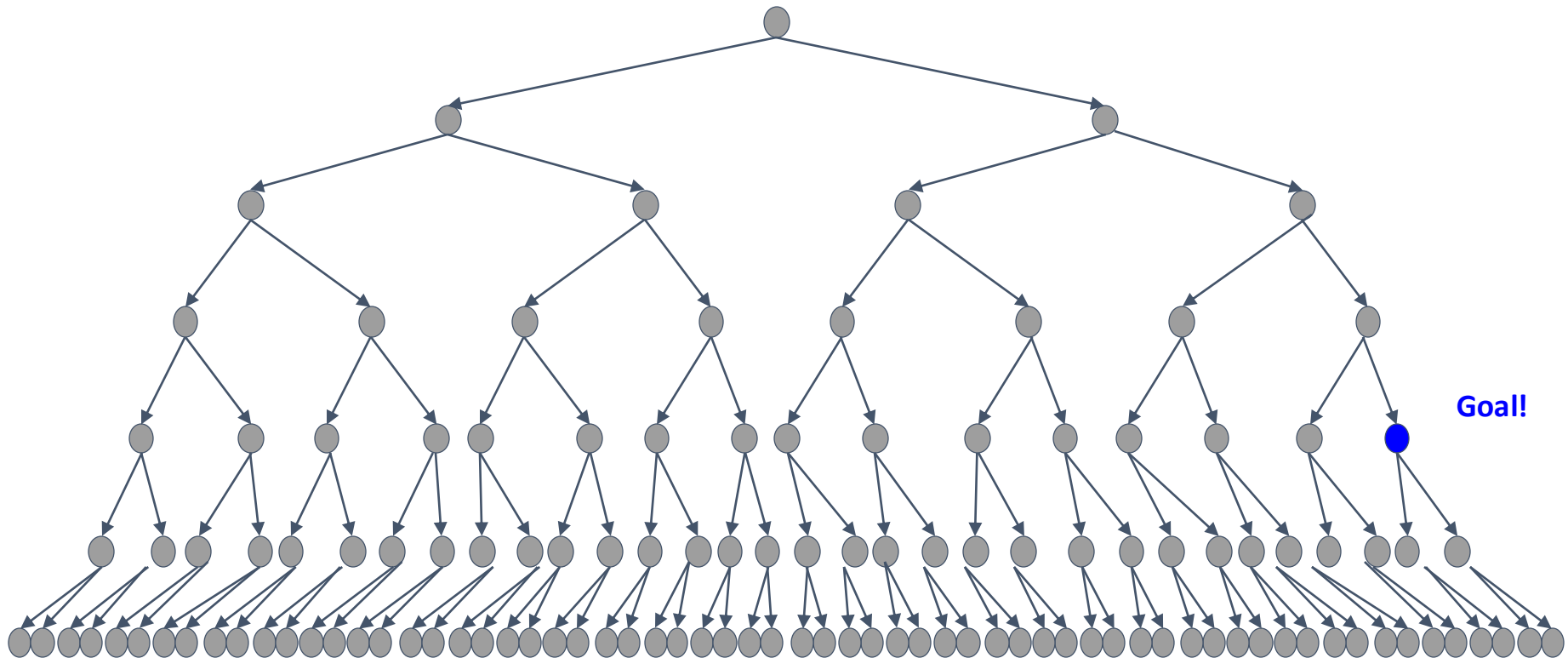
b $\Rightarrow$ branching factor

d $\Rightarrow$ depth of shallowest solution

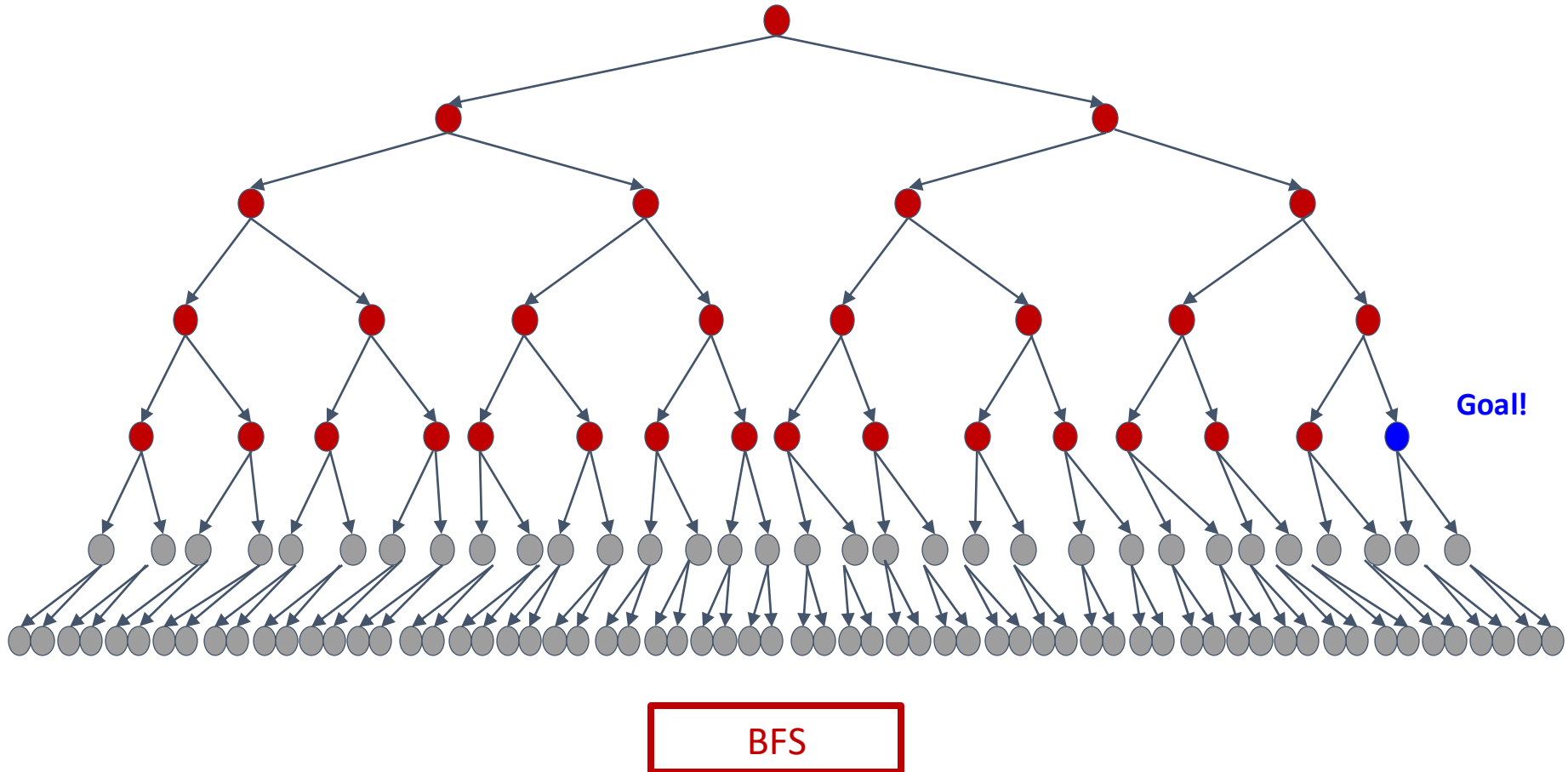
m $\Rightarrow$ maximum depth of the search tree

	BFS	DFS
Completeness (finite “b”)	<i>Yes</i>	<i>No</i>
Optimality (step costs=1)	<i>Yes</i>	<i>No</i>
Optimality (general costs)	<i>No</i>	<i>No</i>
Space Complexity	$O(b^d)$	$O(bm)$
Time Complexity	$O(b^d)$	$O(b^m)$

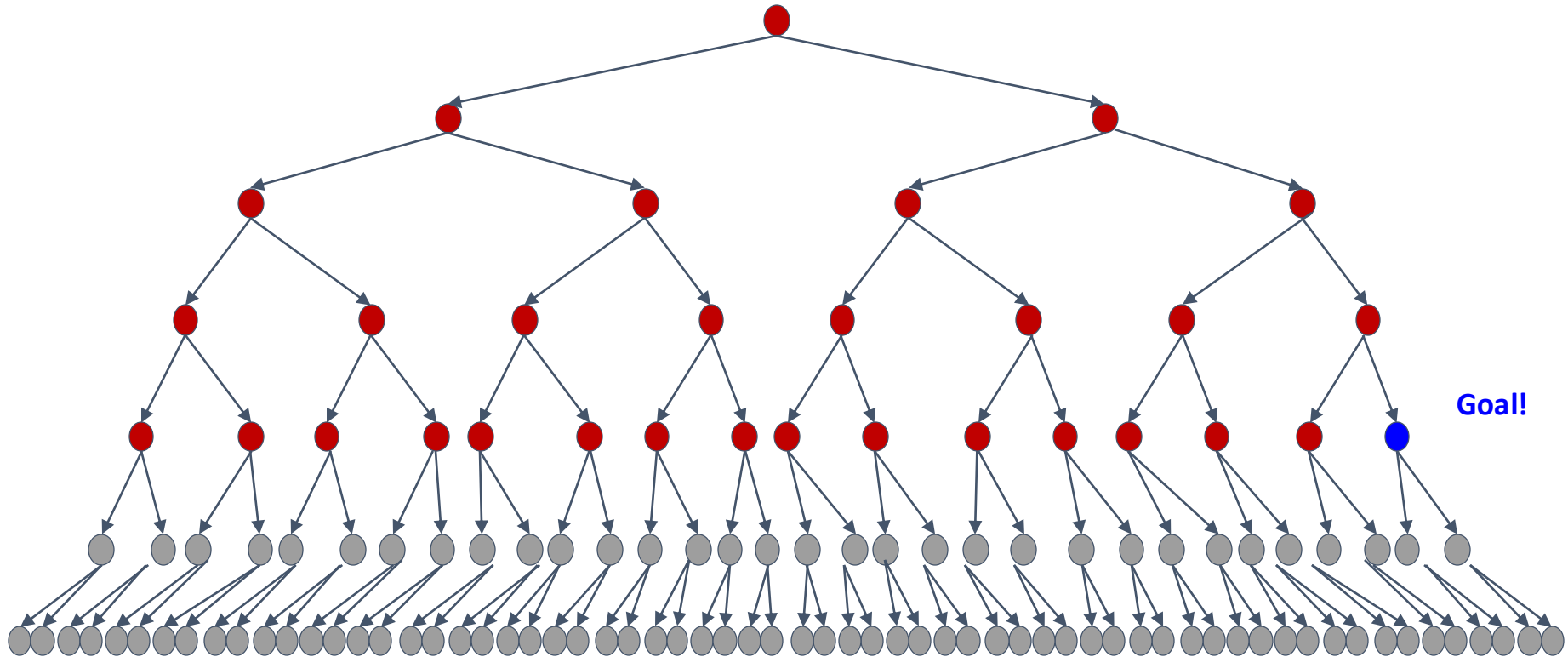
Breadth-First Search vs Depth-First Search



Breadth-First Search vs Depth-First Search



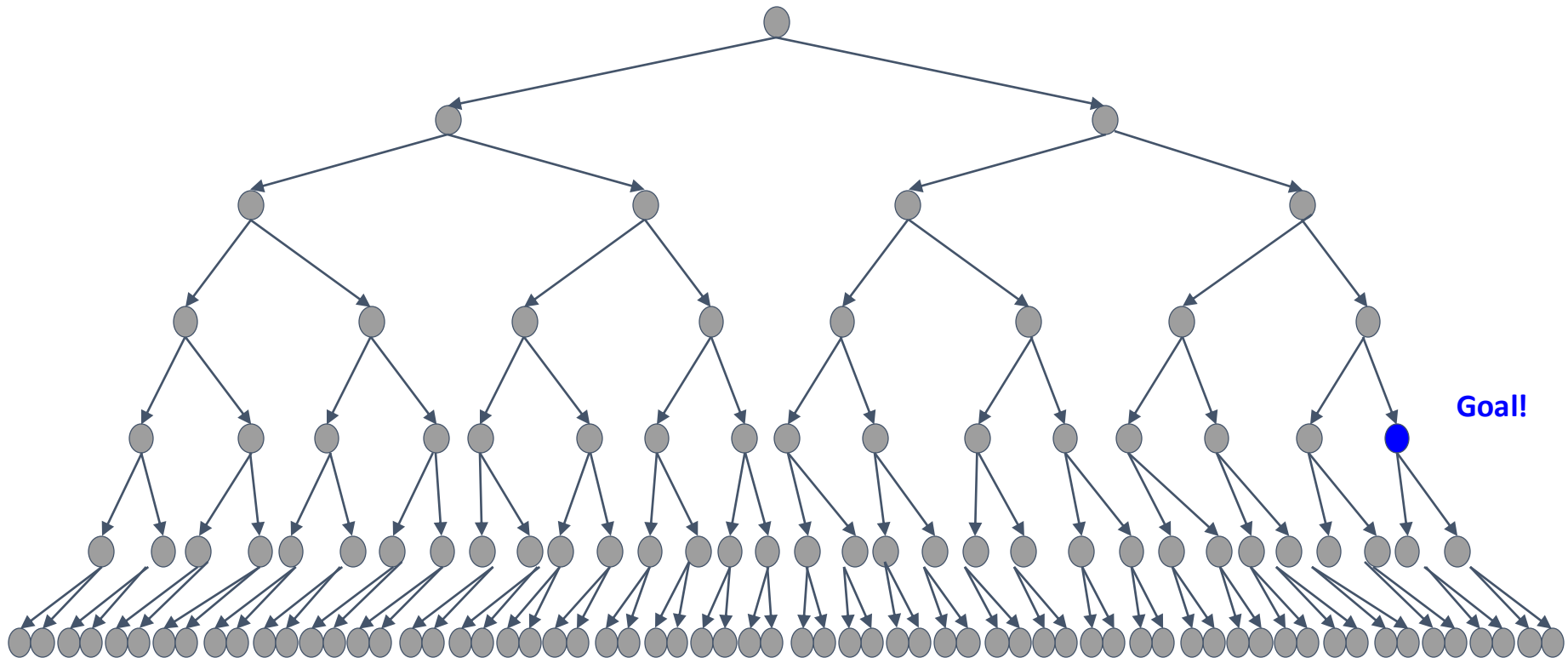
Breadth-First Search vs Depth-First Search



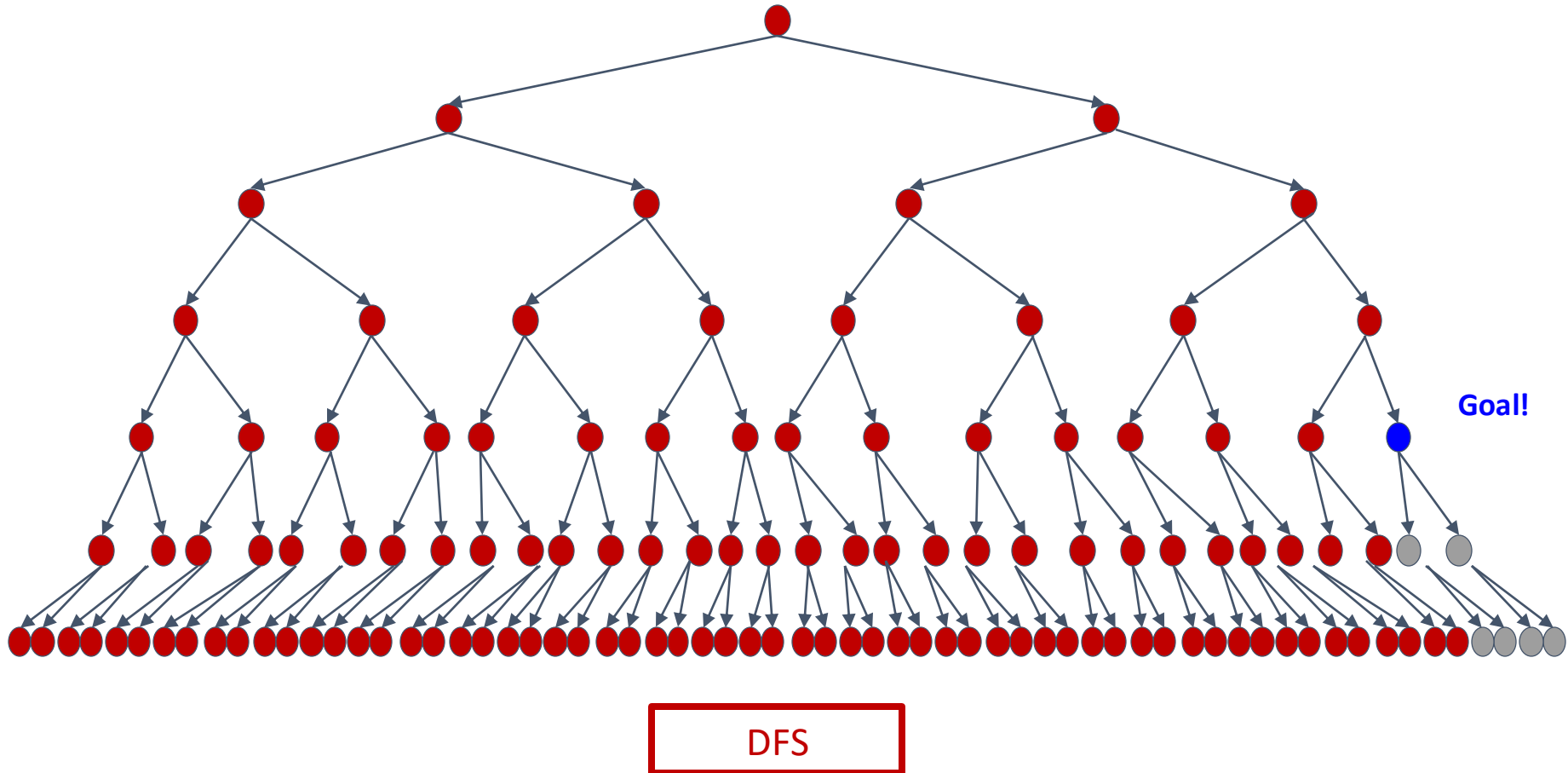
BFS

$$\begin{aligned}\text{Time complexity} &= 1 + 2 + 4 + 8 + 16 \\ &= 2^0 + 2^1 + 2^2 + 2^3 + 2^4 \\ &= O(2^4) \\ &= O(b^d)\end{aligned}$$

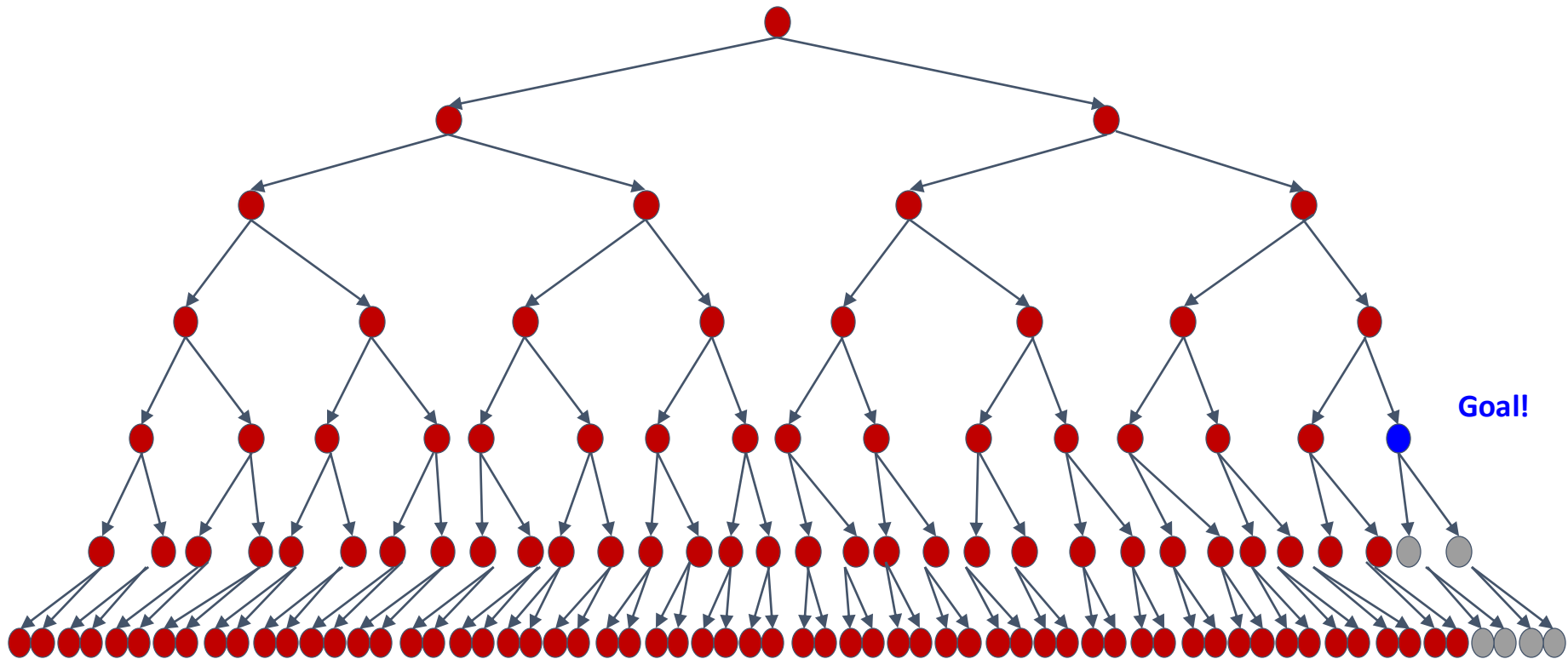
Breadth-First Search vs Depth-First Search



Breadth-First Search vs Depth-First Search



Breadth-First Search vs Depth-First Search



DFS

$$\begin{aligned}\text{Time complexity} &= 1 + 2 + 4 + 8 + 16 + (32-2) + (64-4) \\ &= O(2^6) \\ &= O(b^m)\end{aligned}$$

Key Properties

$b \Rightarrow$ branching factor

$d \Rightarrow$ depth of shallowest solution

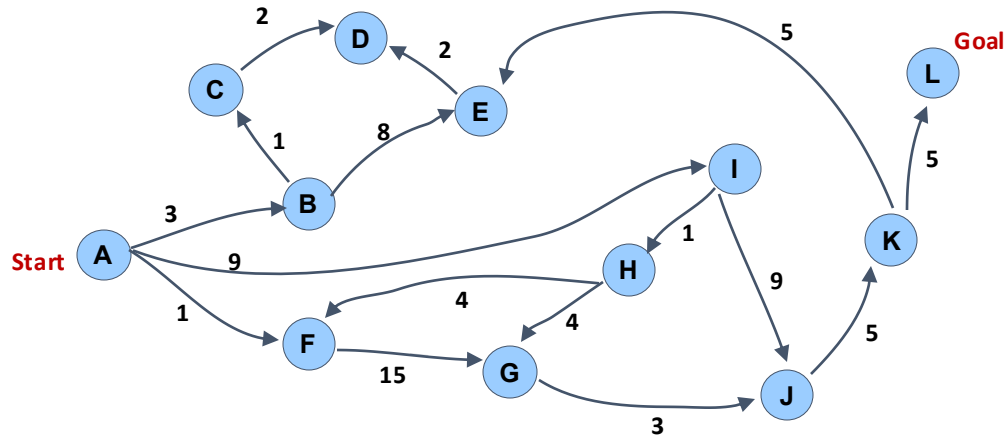
$m \Rightarrow$ maximum depth of the search tree

$\epsilon \Rightarrow$ minimum graph edge cost

$C^* \Rightarrow$ optimal path cost

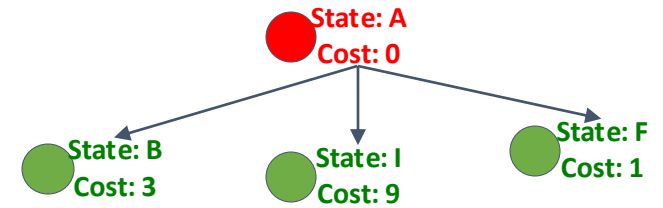
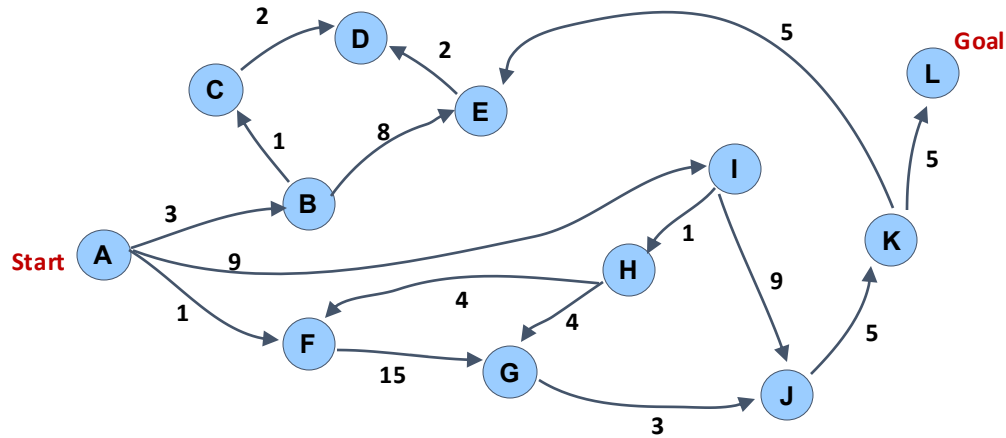
	BFS	DFS	Uniform
Completeness (finite “b”)	Yes	No	Yes*
Optimality (step costs=1)	Yes	No	Yes
Optimality (general costs)	No	No	Yes
Space Complexity	$O(b^d)$	$O(bm)$	$O(b^{C^*/\epsilon})$
Time Complexity	$O(b^d)$	$O(b^m)$	$O(b^{C^*/\epsilon})$

Example: Uniform Cost Search

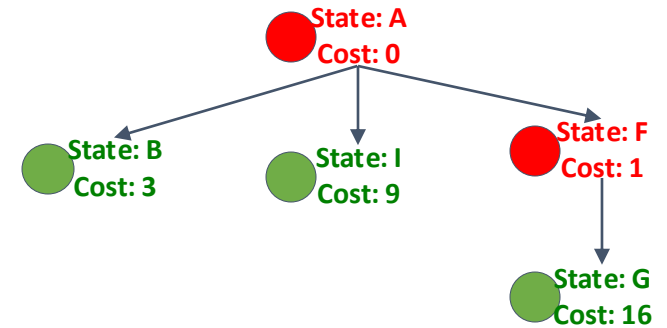
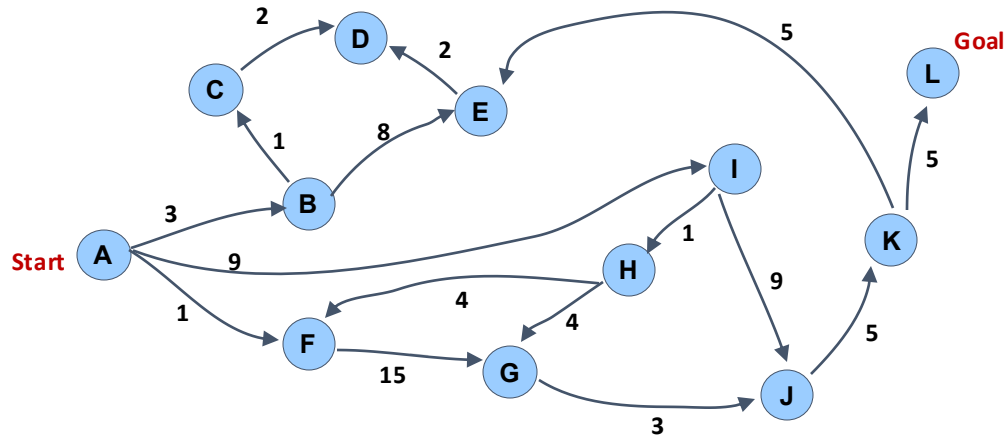


State: A
Cost: 0

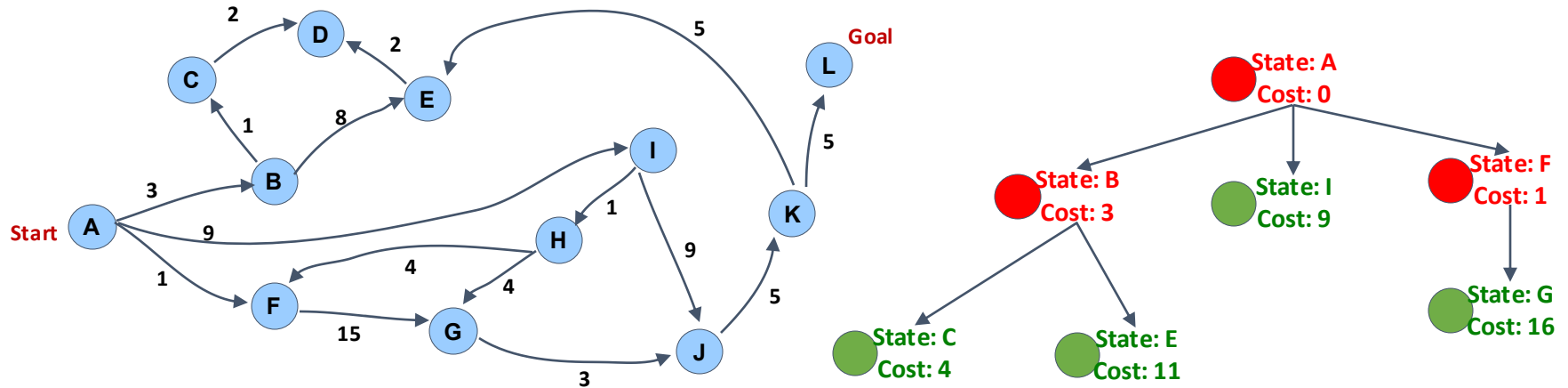
Example: Uniform Cost Search



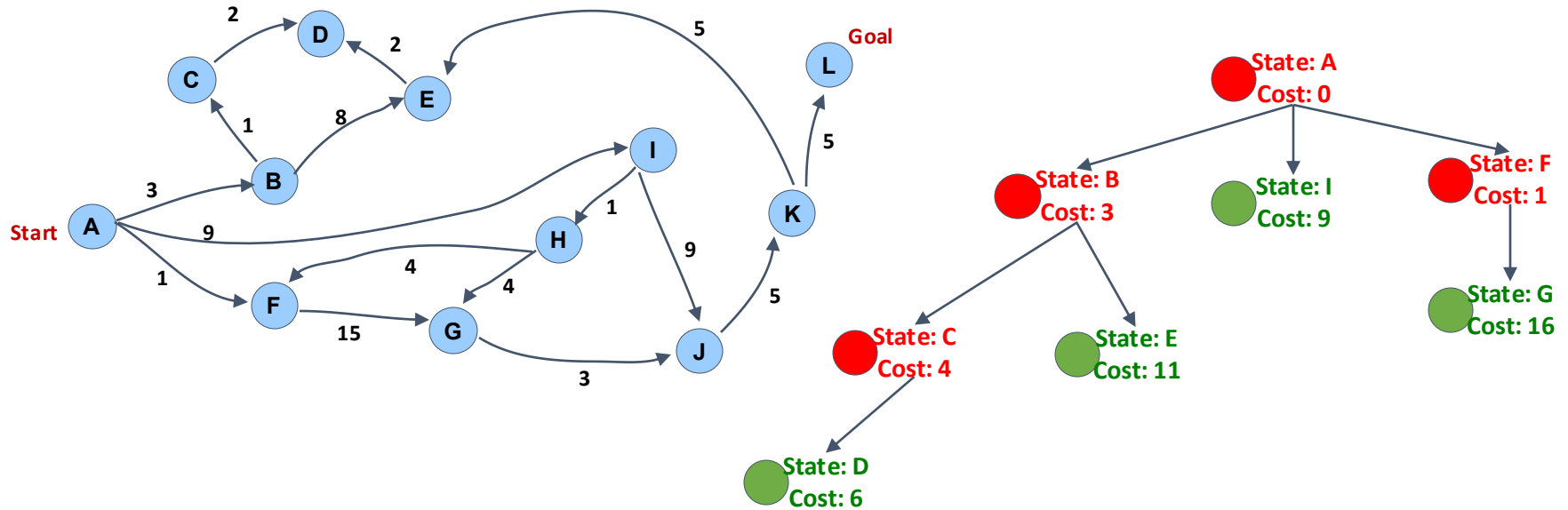
Example: Uniform Cost Search



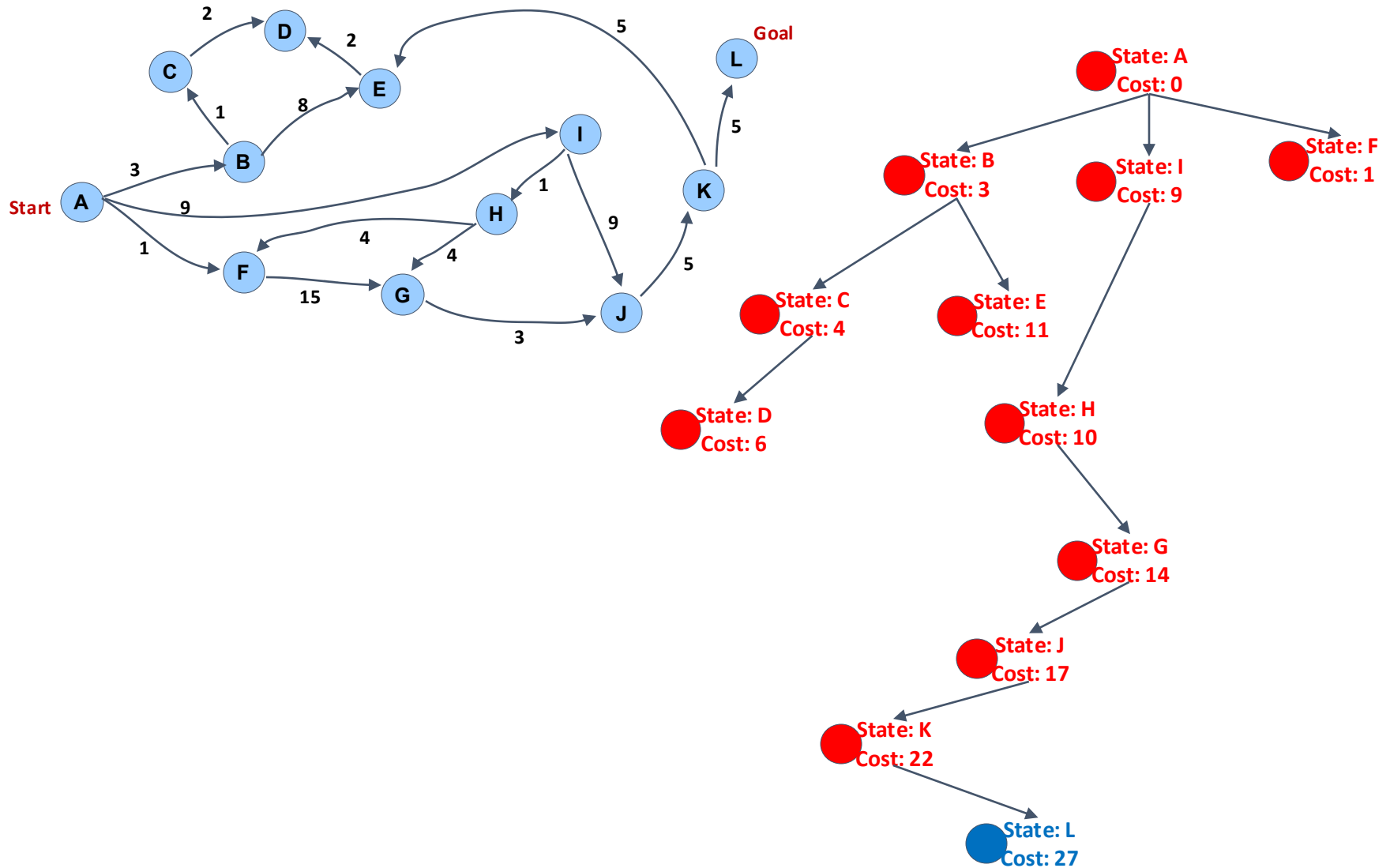
Example: Uniform Cost Search



Example: Uniform Cost Search



Example: Uniform Cost Search



Key Properties

$b \Rightarrow$ branching factor

$d \Rightarrow$ depth of shallowest solution

$m \Rightarrow$ maximum depth of the search tree

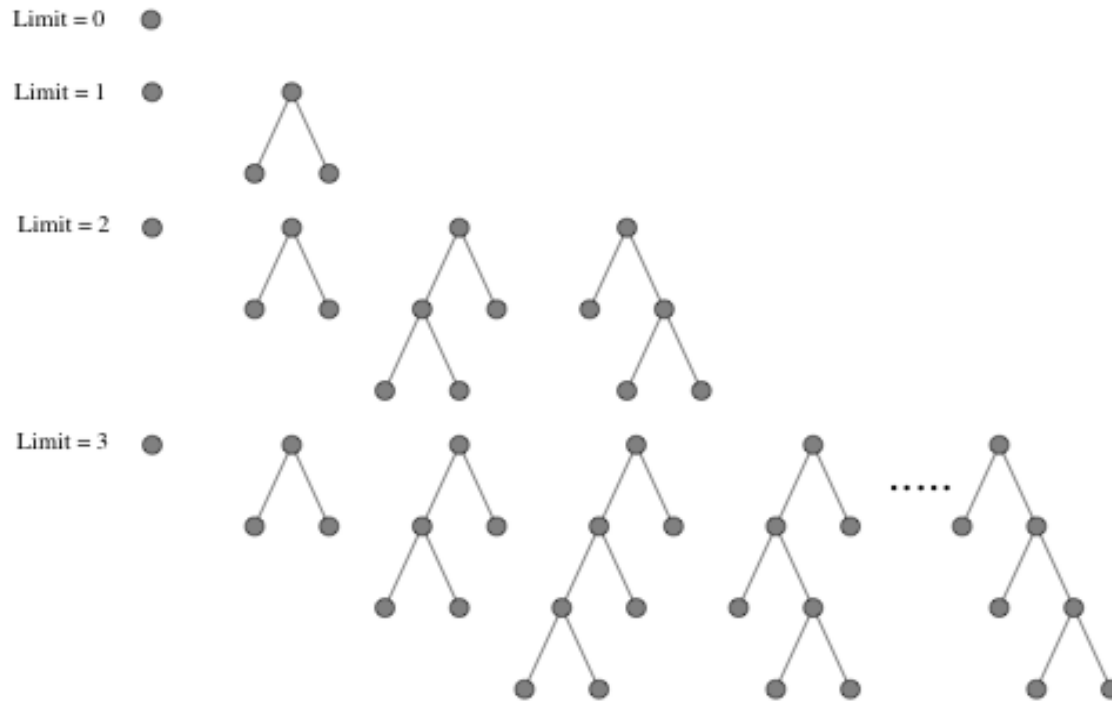
$\epsilon \Rightarrow$ minimum graph edge cost

$C^* \Rightarrow$ optimal path cost

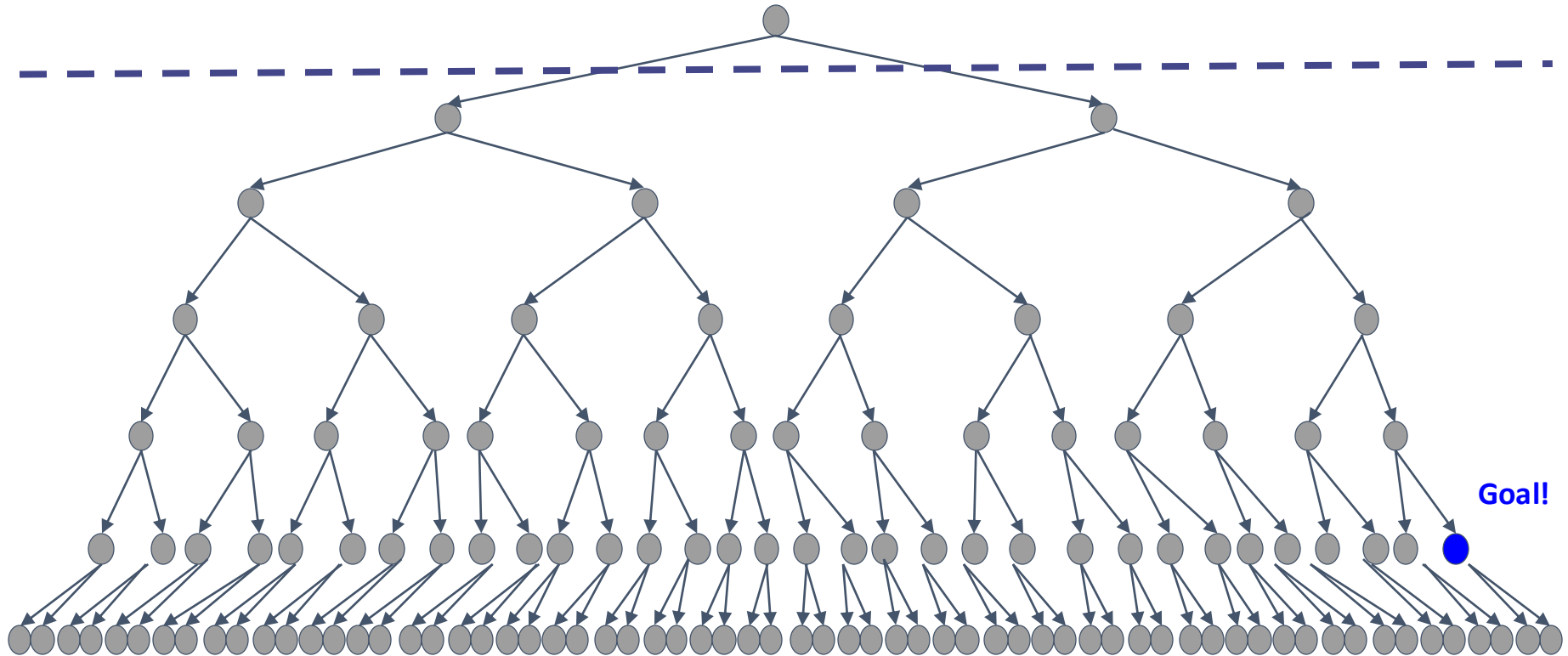
	BFS	DFS	Uniform	IDS
Completeness (finite “b”)	Yes	No	Yes*	Yes
Optimality (step costs=1)	Yes	No	Yes	Yes
Optimality (general costs)	No	No	Yes	No
Space Complexity	$O(b^d)$	$O(bm)$	$O(b^{C^*/\epsilon})$	$O(bd)$
Time Complexity	$O(b^d)$	$O(b^m)$	$O(b^{C^*/\epsilon})$	$O(b^d)$

Iterative Deepening Search

Algorithm: Do depth-limited search, but with increasing depth.



Iterative Deepening Search



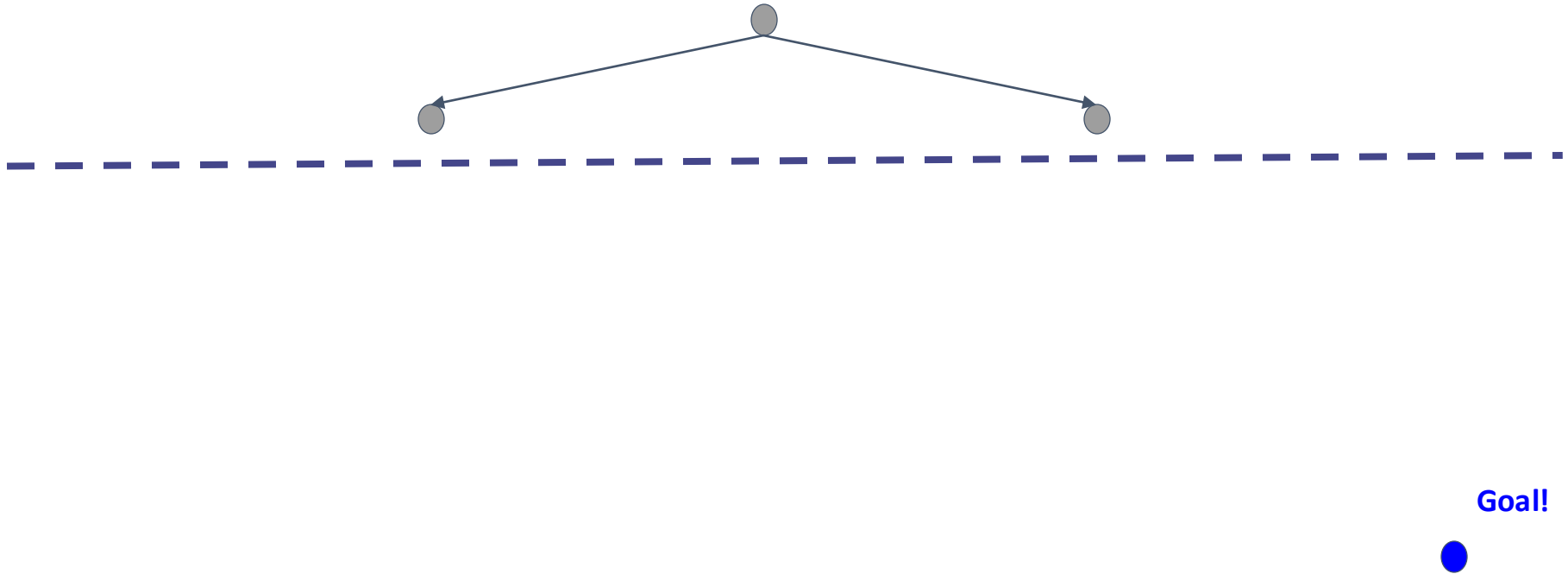
Iterative Deepening Search



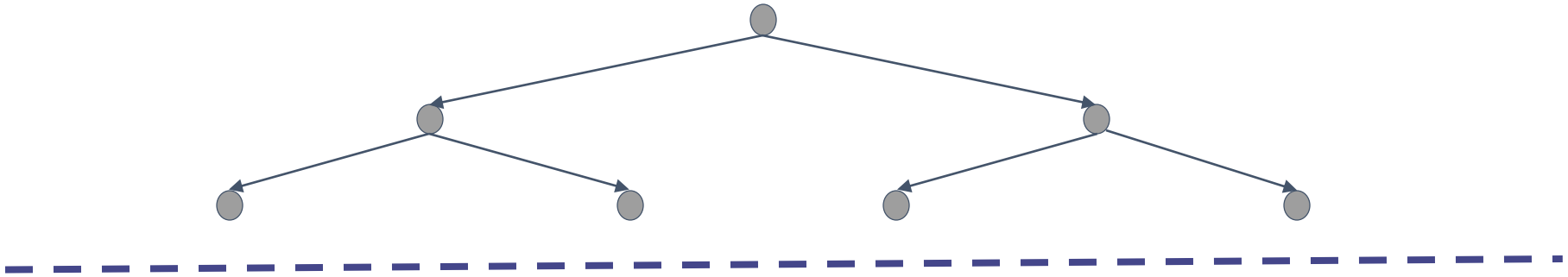
Goal!



Iterative Deepening Search



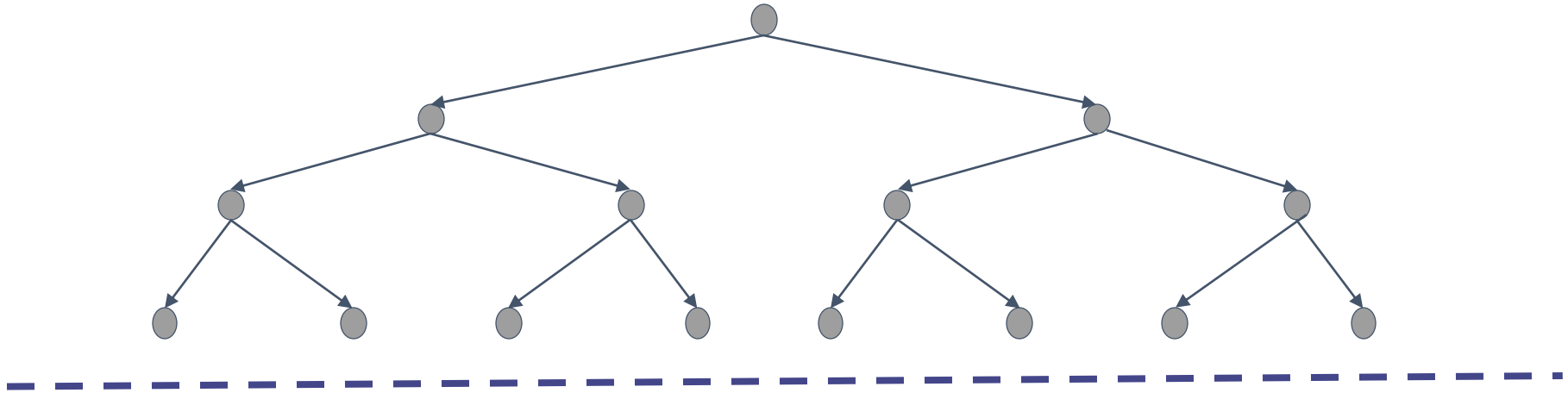
Iterative Deepening Search



Goal!



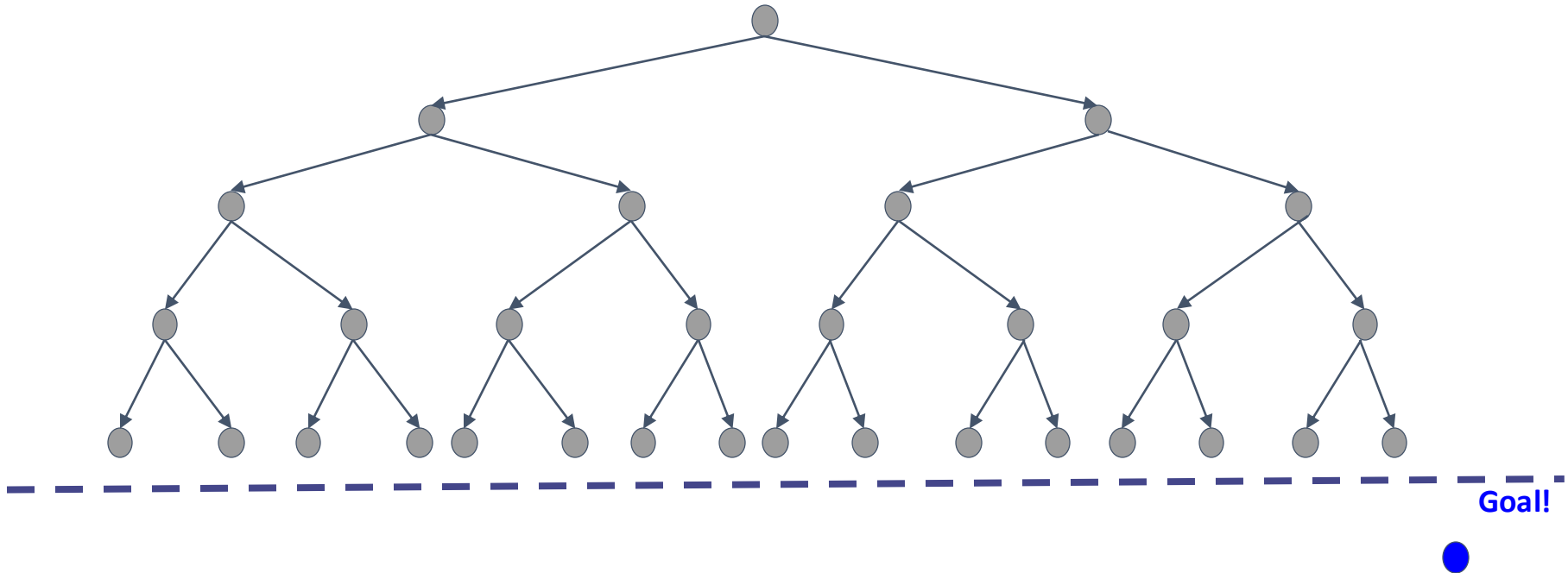
Iterative Deepening Search



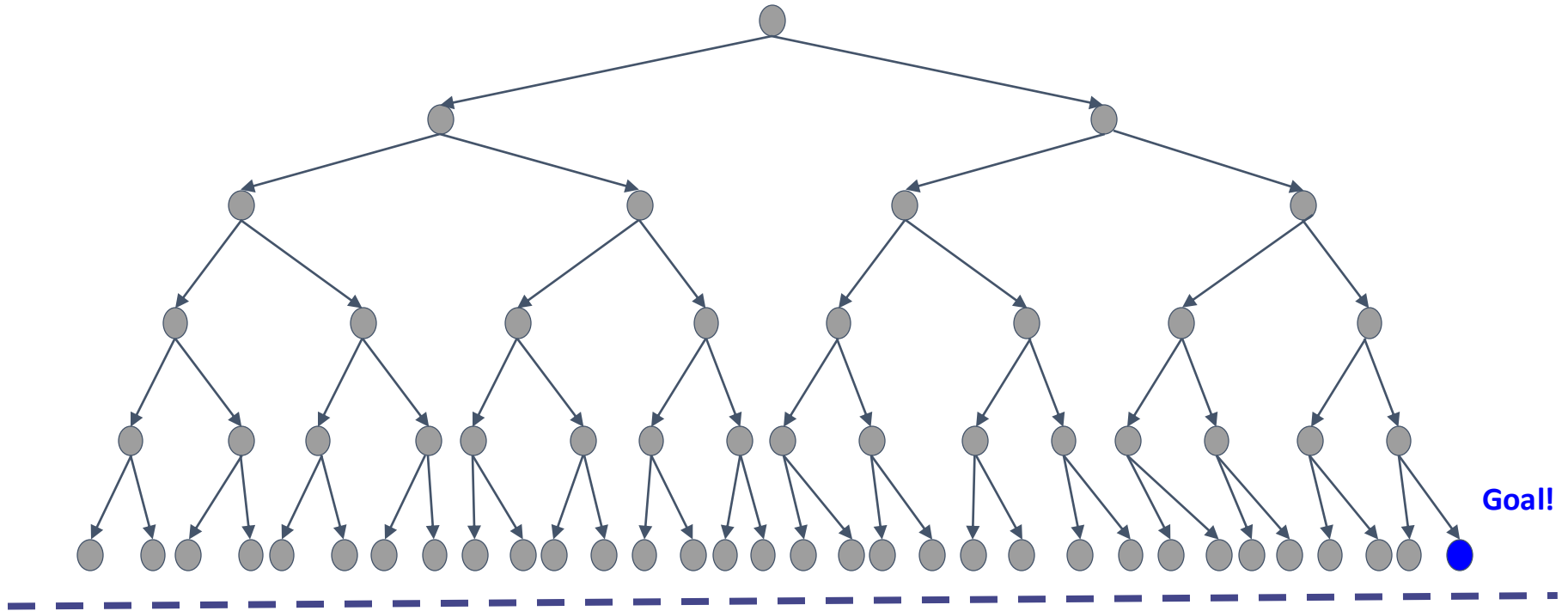
Goal!



Iterative Deepening Search



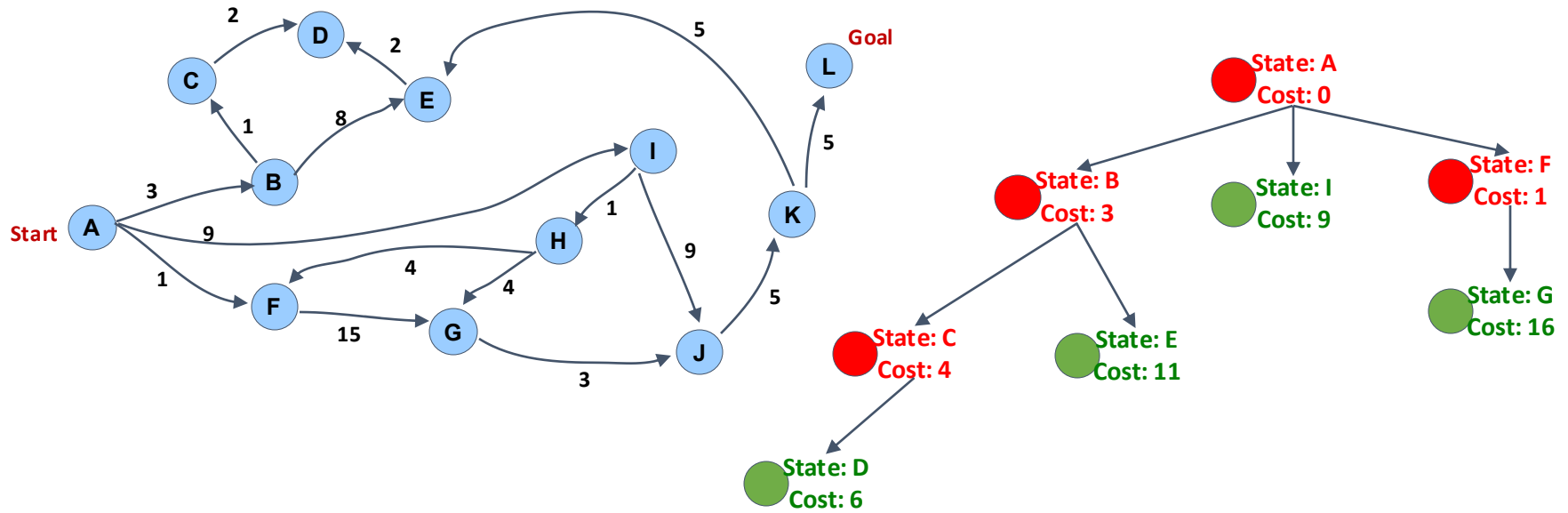
Iterative Deepening Search



Questions

- Iterative Deepening Search:
 - Why isn't the time complexity larger? Isn't there a lot of repetition?
- Uniform Cost Search:
 - Why the asterisk for completeness?
 - What happens to optimality if edge weights are negative?
 - Can this be fixed by adding a positive constant to all edges?
 - Why is space complexity the same as time complexity?
 - What's the relationship between UCS and BFS?
 - Why is uniform cost search called "uniform"?

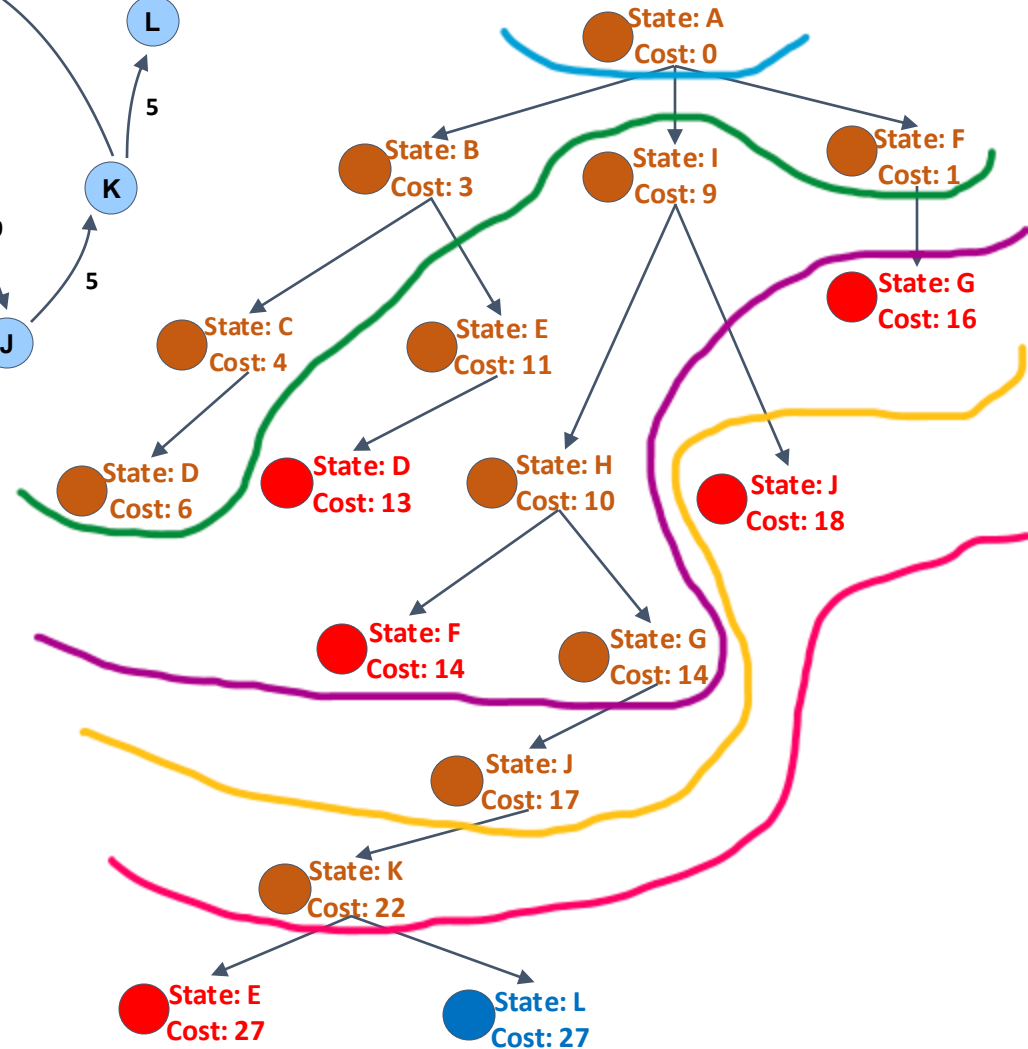
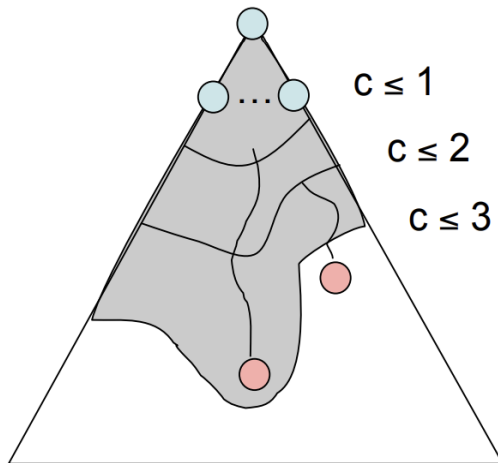
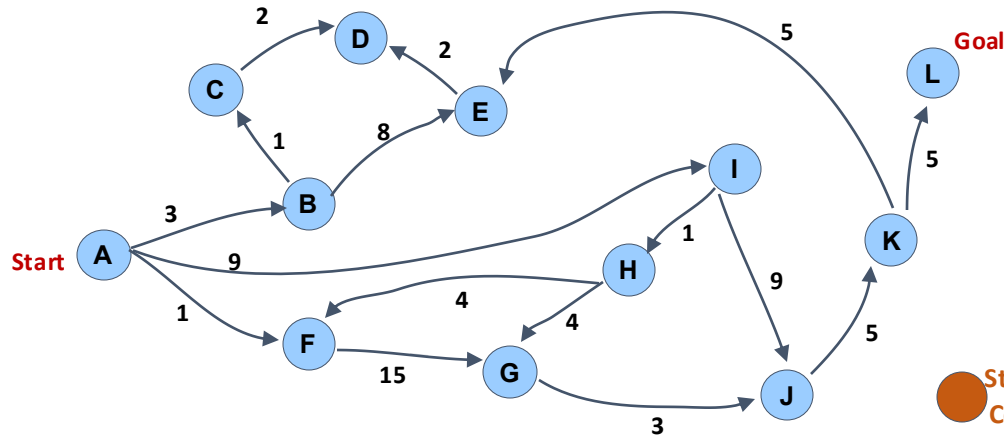
Example: Shortest Path



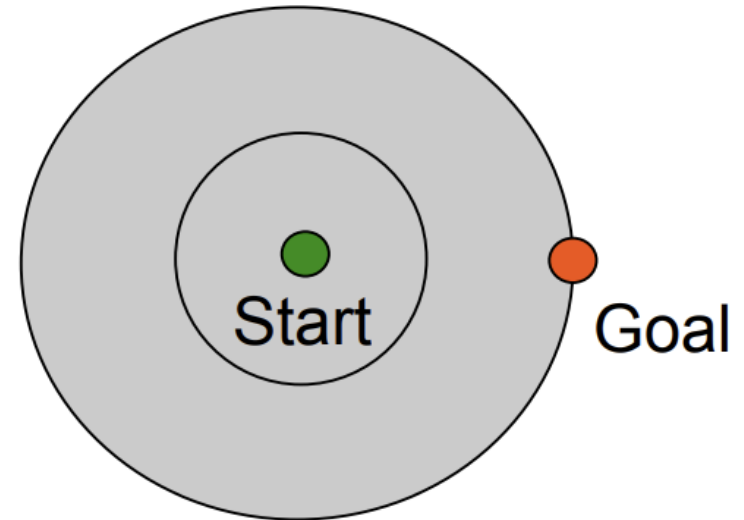
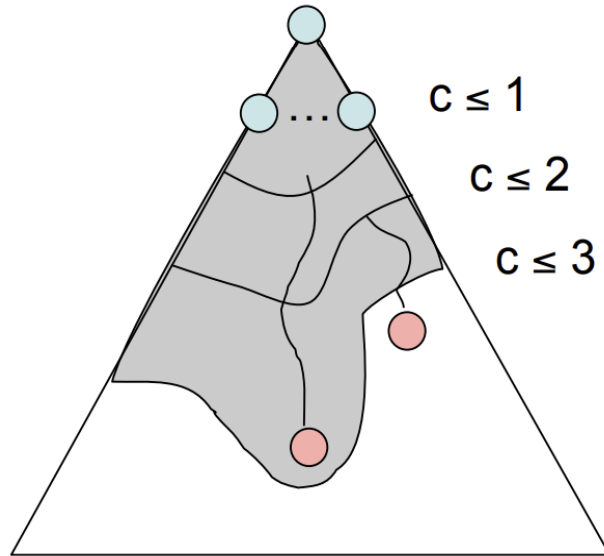
Questions

- Iterative Deepening Search:
 - Why isn't the time complexity larger? Isn't there a lot of repetition?
- Uniform Cost Search:
 - Why the asterisk for completeness?
 - What happens to optimality if edge weights are negative?
 - Can this be fixed by adding a positive constant to all edges?
 - Why is space complexity the same as time complexity?
 - What's the relationship between UCS and BFS?
 - Why is uniform cost search called "uniform"?

Example: Shortest Path

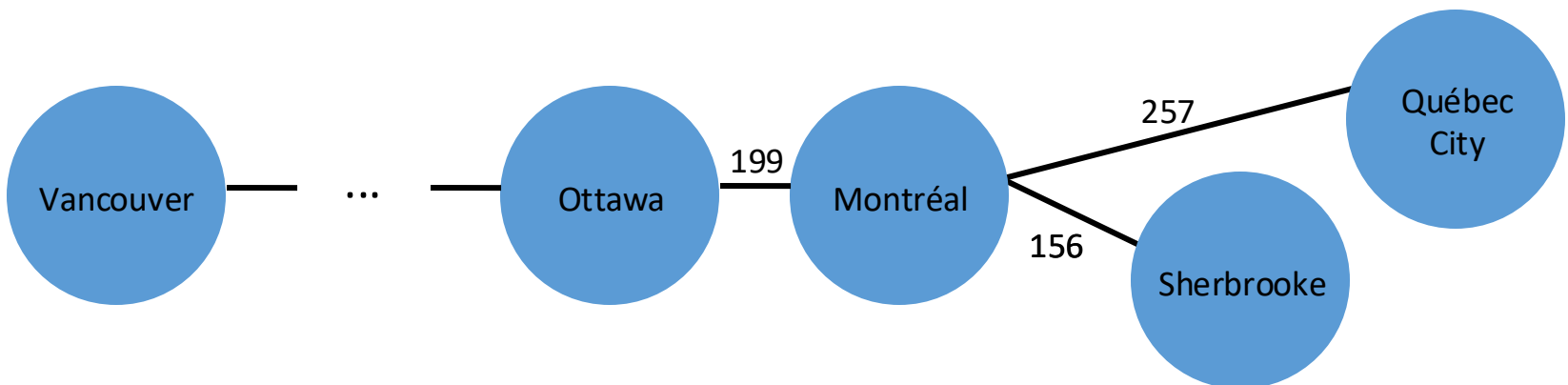


Uninformed Search



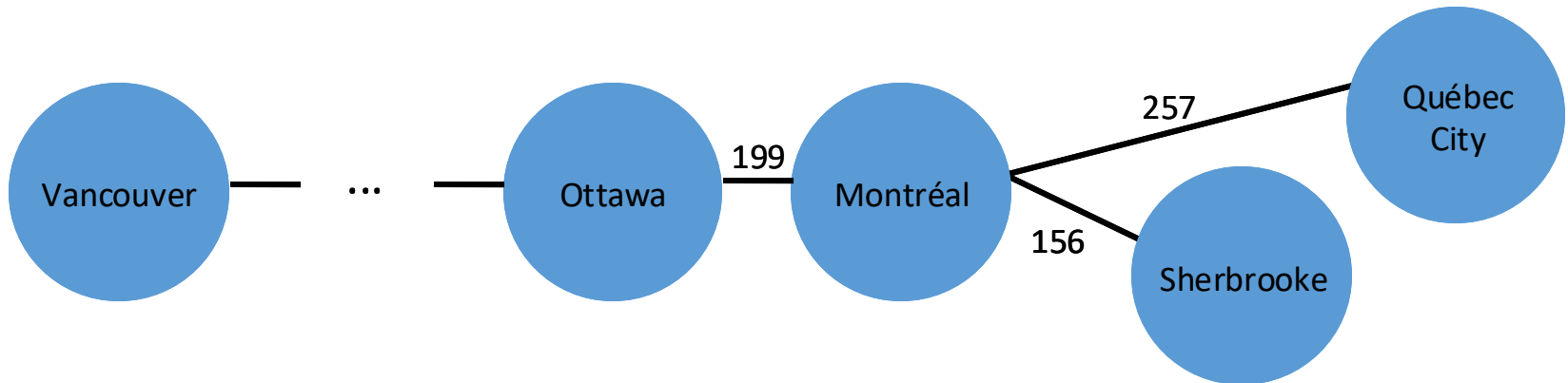
UCS, the good: Complete* and optimal*

UCS, less good: Explores options in every "direction"; no info about goal location



Example Heuristic: Driving to Vancouver

- What is a reasonable heuristic?
 - i.e., What is a reasonable *estimate* of how close a state is to our goal?
 - $h(n)$ = straight-line distance
- Is it always right?
 - No – actual distances driven can be much longer
 - But still helpful!



Example Heuristic: Eight-Puzzle

- What would be a good heuristic for this problem?
 - $h_1(n)$ = number of misplaced tiles
 - $h_2(n)$ = Manhattan distance

Sum of horizontal and vertical moves needed to reach the target position (think moving on city blocks)

5	4	
6	1	8
7	3	2

Start State

1	2	3
8		4
7	6	5

Goal State

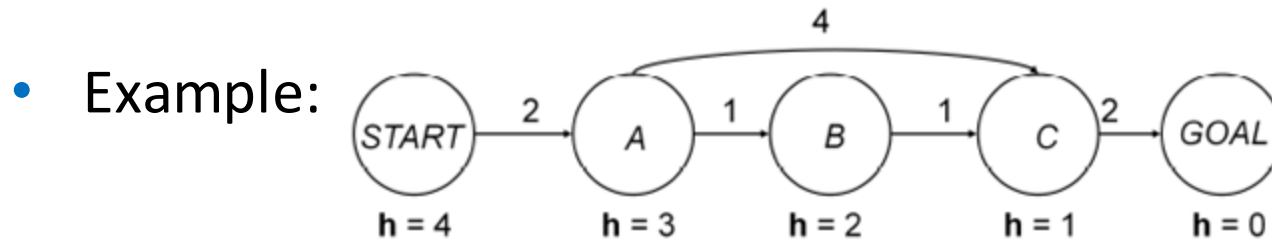
Where do heuristics come from?

Possible strategies:

- Prior knowledge about the problem
- Exact solution to a **relaxed** version of the problem
 - E.g., If rules of eight-puzzle are changed to allow a tile to move anywhere. Then $h_1(n)$ gives the exact distance to goal.
 - E.g., If rules of eight-puzzle are changed to allow a tile to move to any adjacent square. Then $h_2(n)$ gives the exact distance to goal.
- Learning from prior experience
 - More on this later

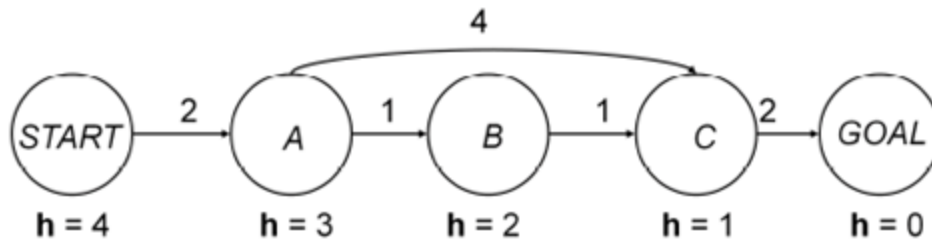
Best-First Search

- **Algorithm:** expand the **most promising node** according to the **heuristic**.



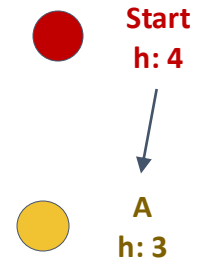
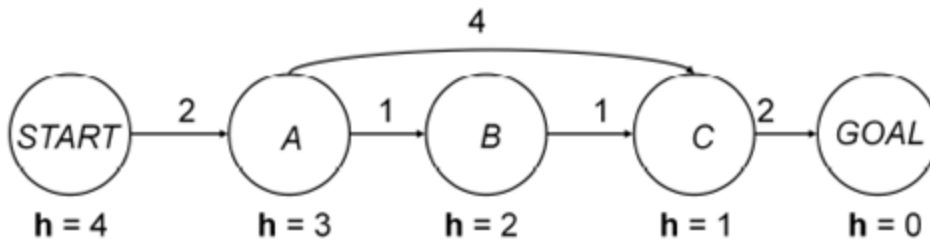
- Numbers on edges indicate the edge/operator cost, notations below the nodes "h=..." give the heuristic value

Best-First Search

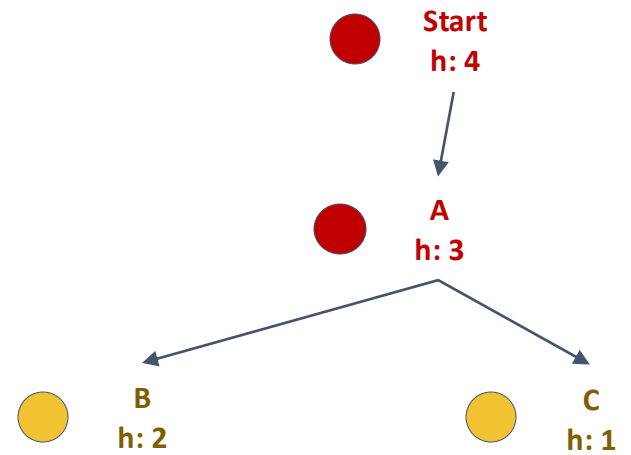
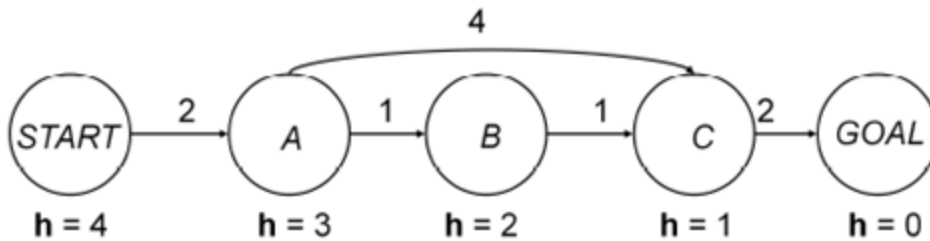


Start
h: 4

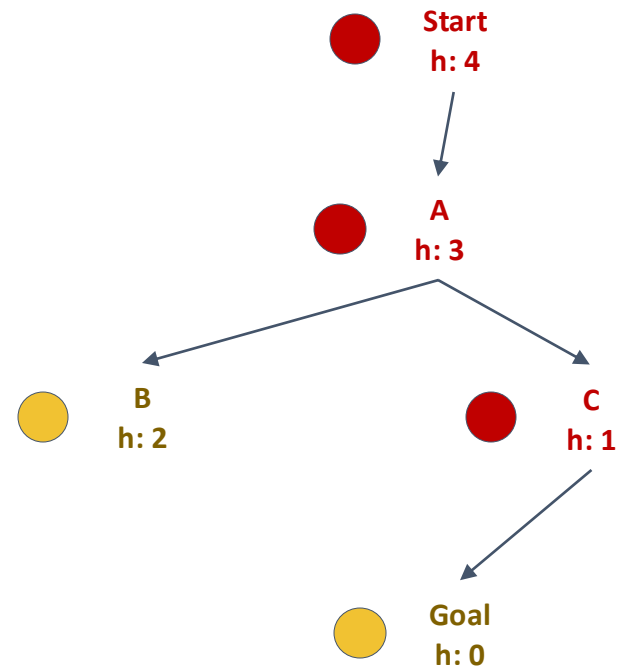
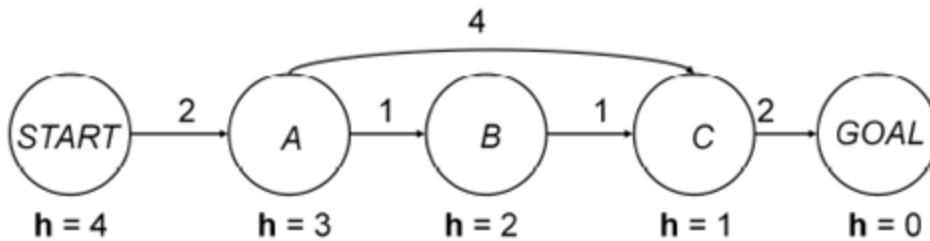
Best-First Search



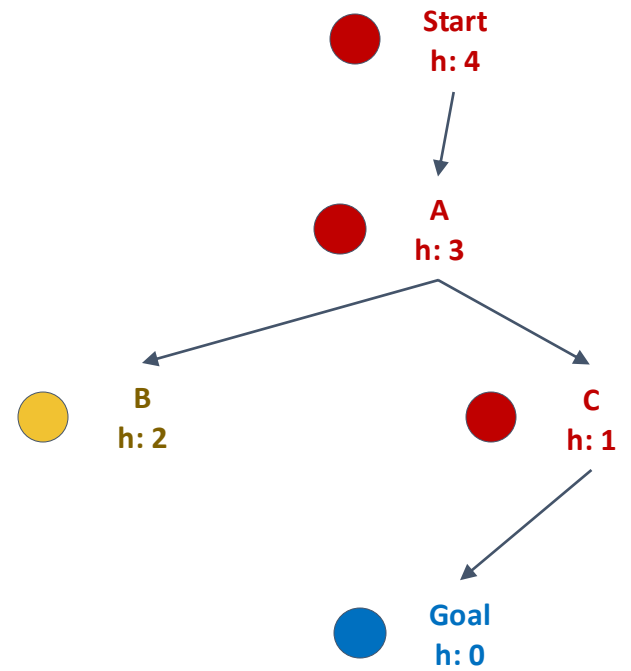
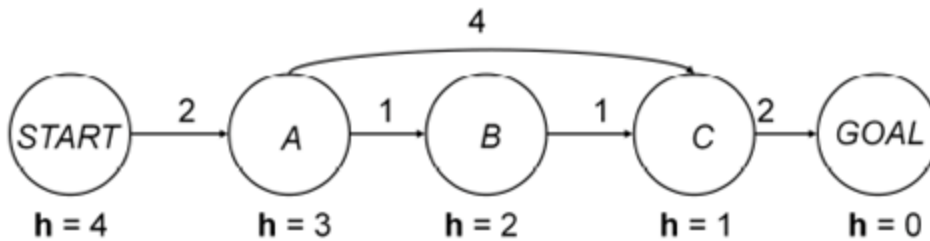
Best-First Search



Best-First Search

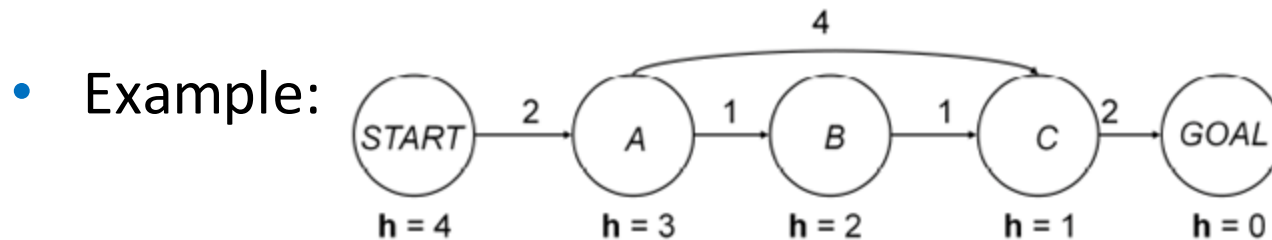


Best-First Search



Best-First Search

- **Algorithm:** expand the **most promising node** according to the **heuristic**.



- What is the sequence of nodes visited in Best-First Search?

START -> A -> C -> GOAL

- What about this?

START -> A -> B -> C -> GOAL

Cost is $2+4+2=8$

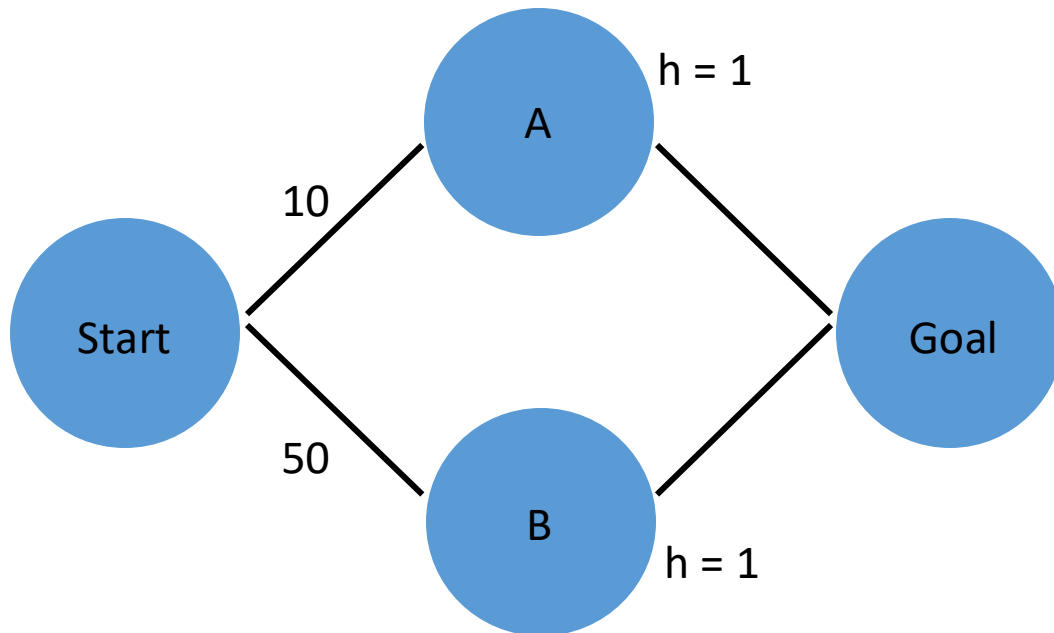
vs

$2+1+1+2=6$

Proof that Best-First Search is not optimal

Fixing Best-First Search

- What's the problem with best-first search?
 - It does not consider the cost so far!!
- Let's use both
 - $g(n)$: Cost of path from the start to the node
 - $h(n)$: Heuristic cost of the path from the node to the goal

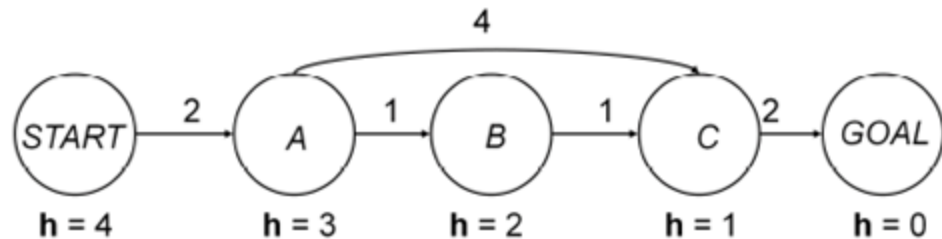


Fixing Best-First Search

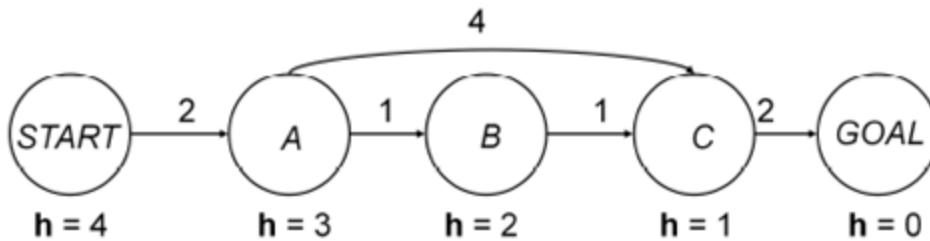
- What's the problem with best-first search?
 - It does not consider the cost so far!!
- Let's use both
 - $g(n)$: Cost of path from the start to the node
 - $h(n)$: Heuristic cost of the path from the node to the goal
- **A* search:** greedy with respect to $f(n) = g(n) + h(n)$
 - Using only $g(n) \rightarrow$ Uniform cost search
 - Orders by path cost, or backward cost
 - Using only $h(n) \rightarrow$ Best-first search
 - Orders by distance to goal, or forward cost

A* search

- **Algorithm:**
 - At every step, take node n from the front of the queue.
 - Add to queue successors n' with **priorities**: $f(n') = g(n') + h(n')$
 - Terminate when a goal state is popped from the queue.
- Previous example:



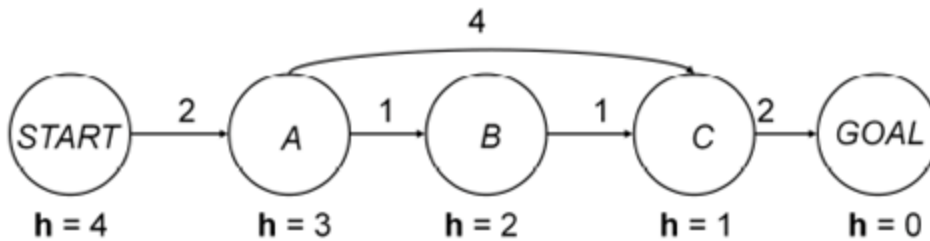
A* search



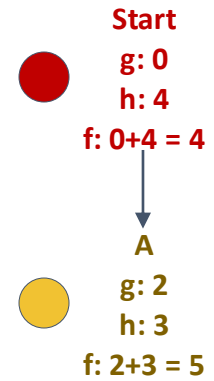
Start
g: 0
h: 4
f: 0+4 = 4

$$f(n) = g(n) + h(n)$$

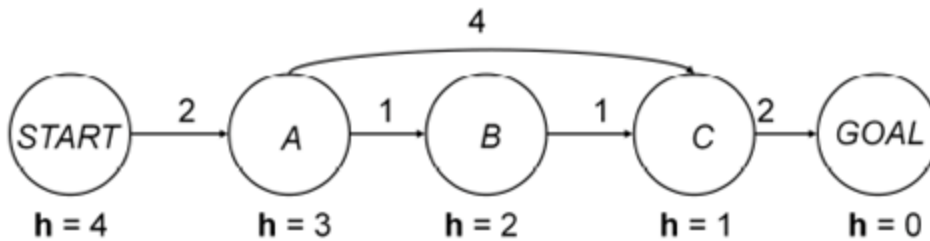
A* search



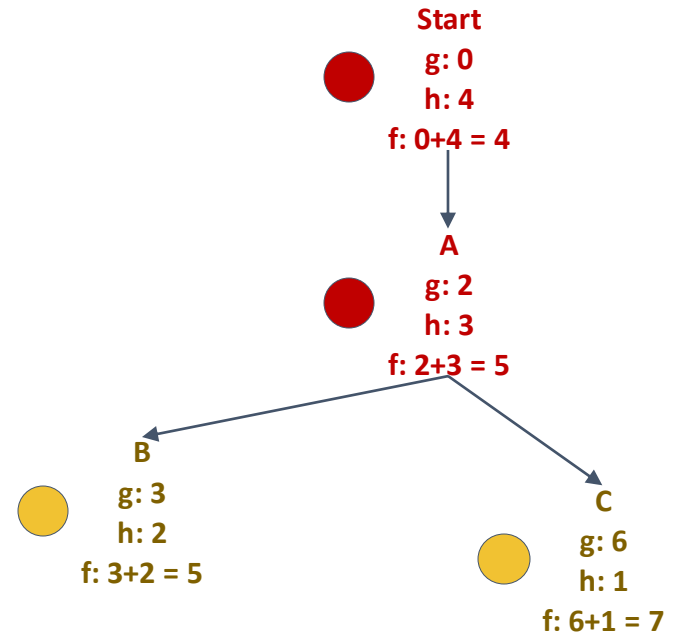
$$f(n) = g(n) + h(n)$$



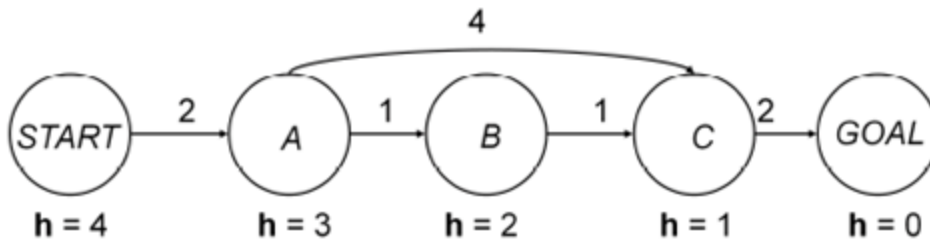
A* search



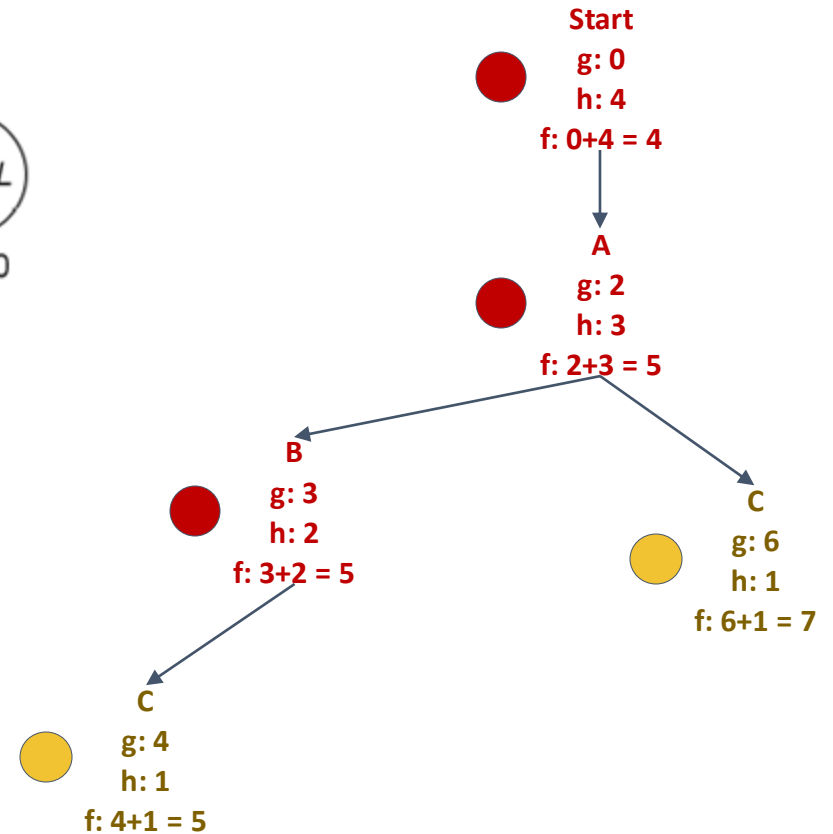
$$f(n) = g(n) + h(n)$$



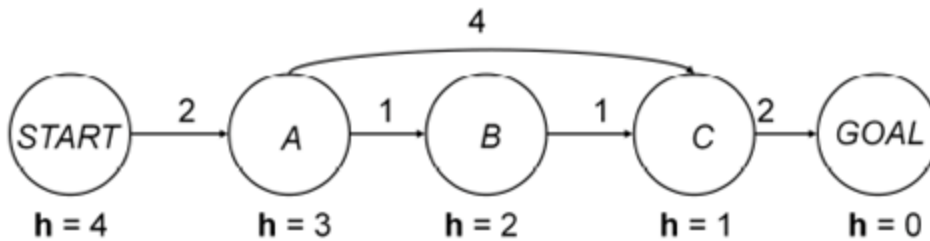
A* search



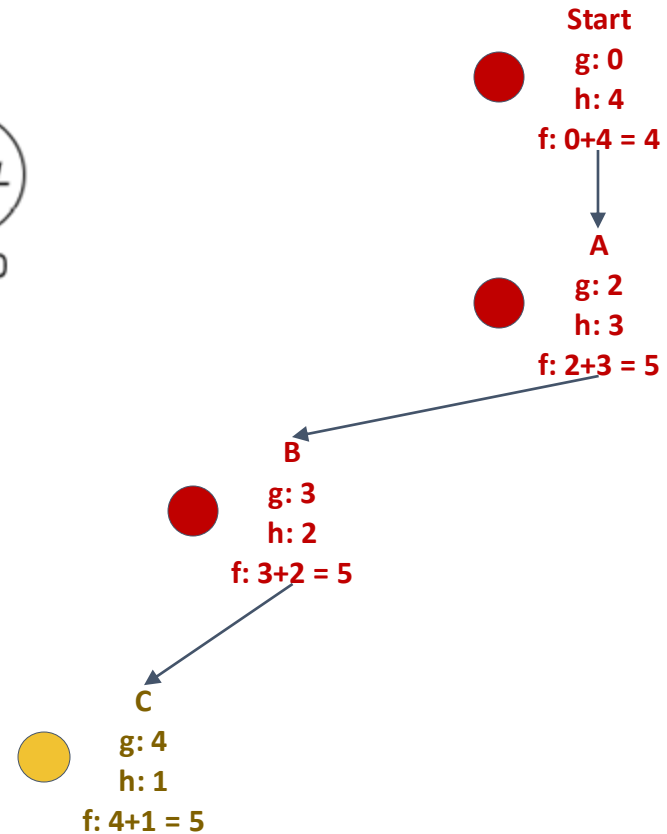
$$f(n) = g(n) + h(n)$$



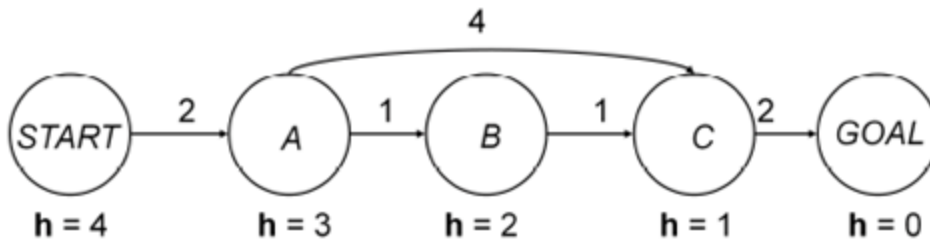
A* search



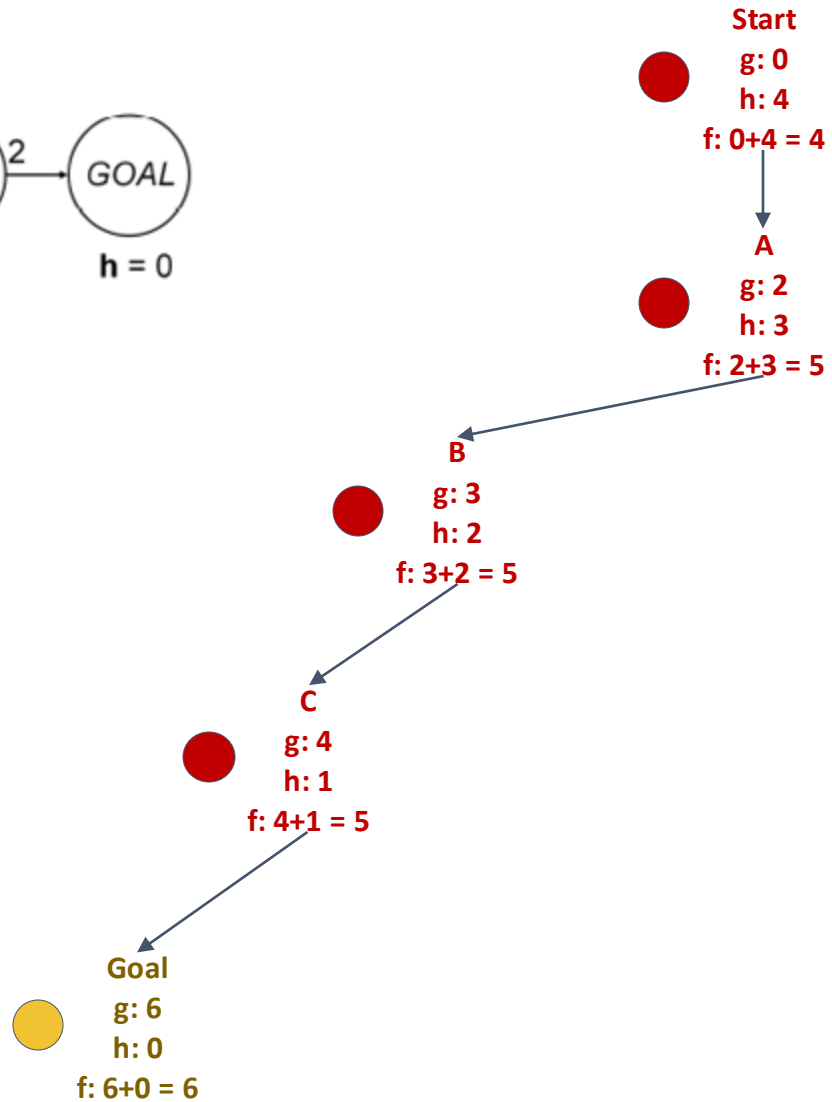
$$f(n) = g(n) + h(n)$$



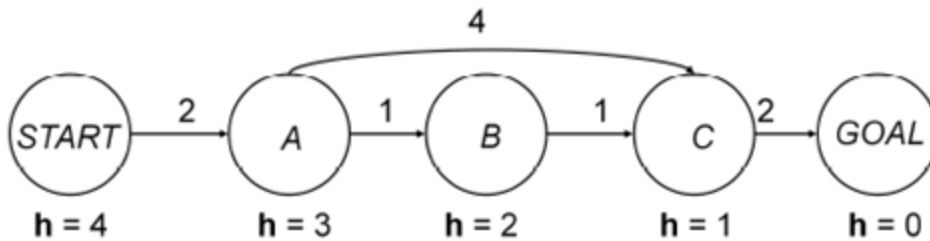
A* search



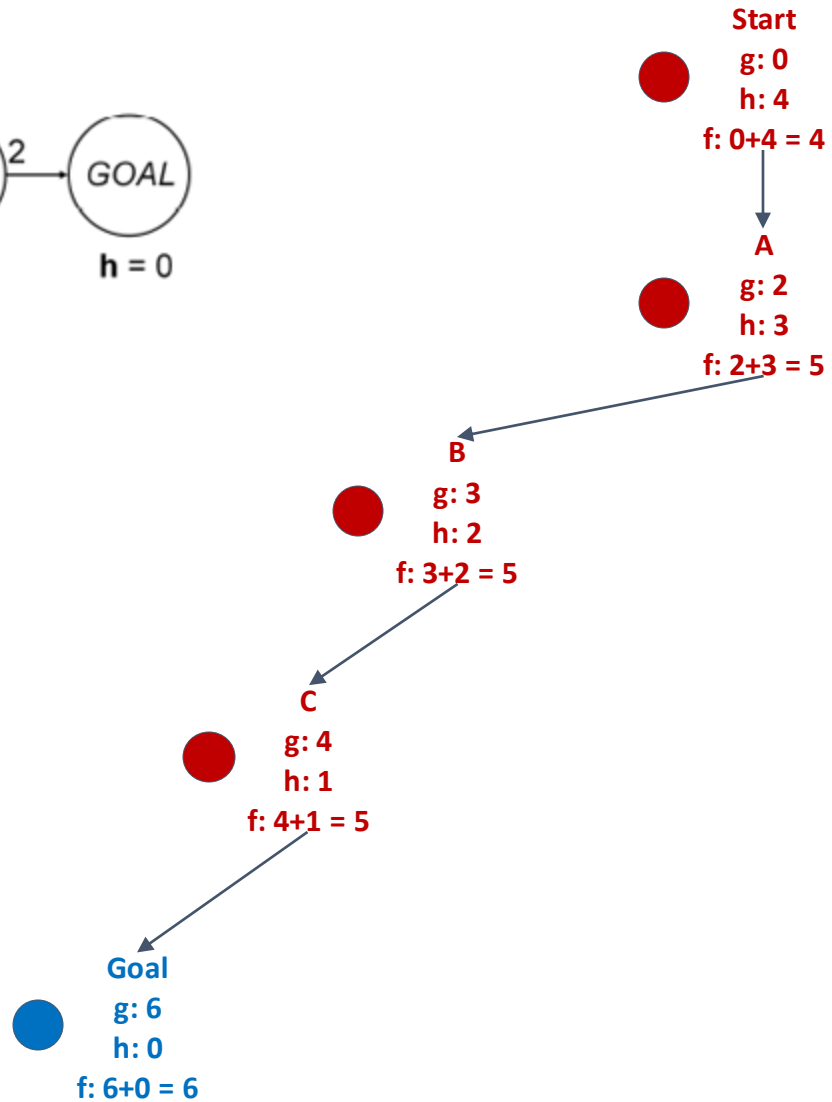
$$f(n) = g(n) + h(n)$$



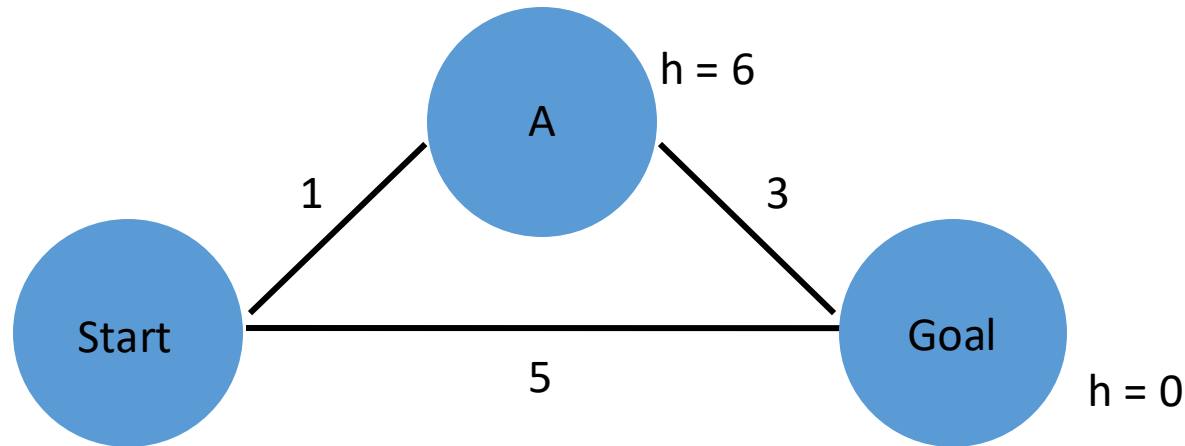
A* search



$$f(n) = g(n) + h(n)$$



When Is A* Optimal?



- **Optimal path:** Start $\rightarrow$ A $\rightarrow$ Goal
- **Path from A*:** Start $\rightarrow$ Goal
- What went wrong?
 - Estimate of good path cost ($1 + 6$) $>$ actual bad path cost (5)
- Need $h(A)$ to be no greater than $cost(A, Goal)$

Properties of Heuristic

- **Admissibility:** The heuristic is always 'optimistic'
 $h(s) \leq h^*(s) \forall s$, where $h^*(s)$ is the true cost to the nearest goal
 - i.e., it never overestimates
 - Driving to Vancouver: is the straight-line distance admissible?
- **Consistency:** The heuristic gets more 'precise' as it gets closer to the goal
 $h(s) \leq c(s, s') + h(s') \forall s, s'$ where $\exists$ edge from s to s'

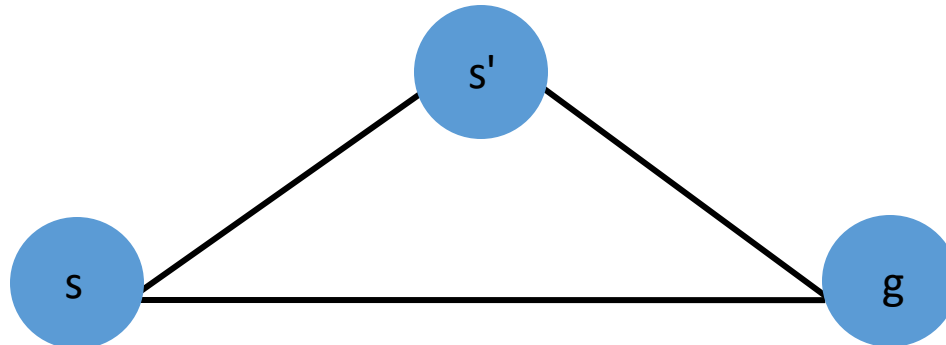
Properties of Heuristic

- **Admissibility:** The heuristic is always 'optimistic'
 $h(s) \leq h^*(s) \forall s$, where $h^*(s)$ is the true cost to the nearest goal

Needed for optimality

- **Consistency:** The heuristic gets more 'precise' as it gets closer to the goal

$h(s) \leq c(s, s') + h(s') \forall s, s'$ where $\exists$ edge from s to s'



Properties of Heuristic

- **Admissibility:** The heuristic is always 'optimistic'
 $h(s) \leq h^*(s) \forall s$, where $h^*(s)$ is the true cost to the nearest goal

Needed for optimality

- **Consistency:** The heuristic gets more 'precise' as it gets closer to the goal

$h(s) \leq c(s, s') + h(s') \forall s, s'$ where $\exists$ edge from s to s'

- This means that for a graph with all costs $c(s, s') > 0$, f cannot decrease along any path:

$$f(s) = g(s) + h(s) \leq g(s) + c(s, s') + h(s') = f(s')$$

- Stronger than admissibility

Admissibility and Optimality for A*

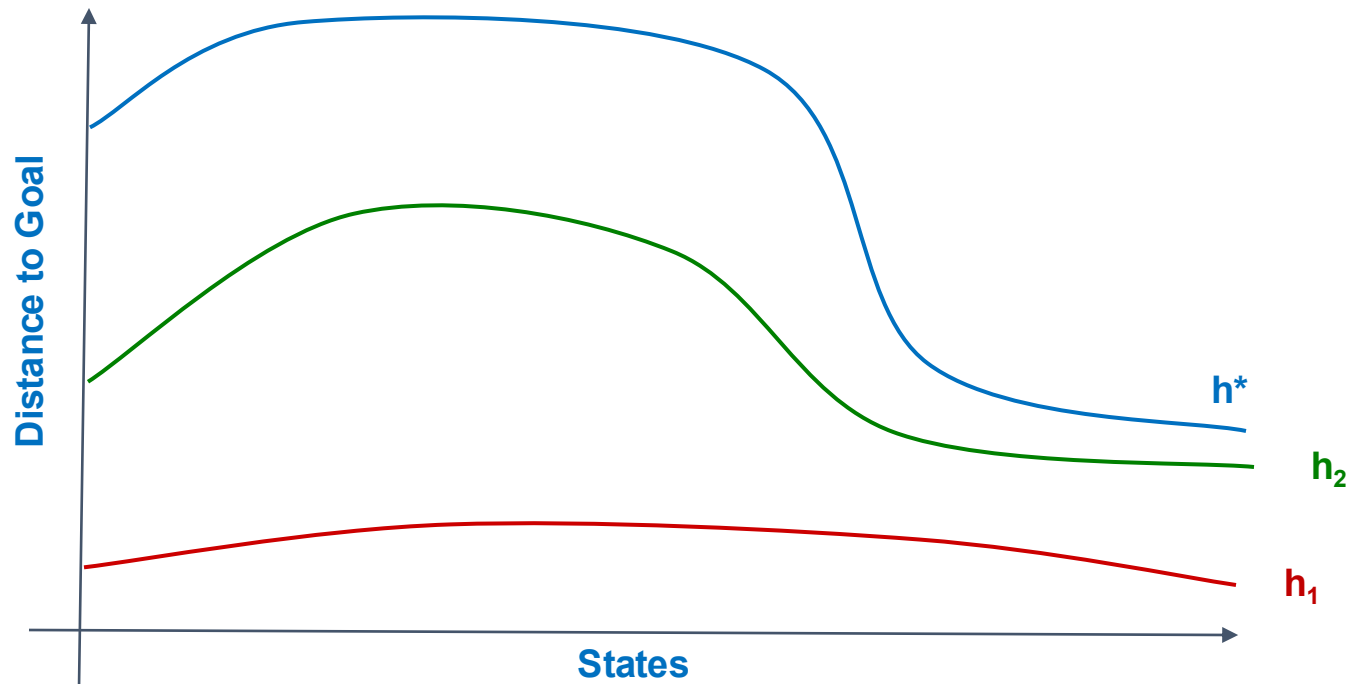
- If A* selects a path p , then p is the lowest-cost path.
- **Proof by contradiction.** Suppose another path p' is actually the shortest path to a goal.
- Consider the moment just before p is chosen from the frontier. Some part of path p' will also be on the frontier (because p is expanded before p'). Call this partial path p'' .
- Because p is expanded before p'' , $f(p) \leq f(p'')$ -- i.e., $g(p) + h(p) \leq g(p'') + h(p'')$.
- Because p is a goal, $h(p) = 0$. Thus $g(p) \leq g(p'') + h(p'')$.
- Because h is admissible, $g(p'') + h(p'') \leq g(p')$ for any path p' to a goal that extends p'' .
- Thus $g(p) \leq g(p'') + h(p'') \leq g(p')$. This contradicts our assumption that p' is the shortest path.

Statements you can try to prove →

- Consistency is a stronger property than admissibility
- Consistency guarantees completeness (it's not required, but it's an easy way to get it)
- A heuristic created by solving a relaxed version of the problem will always be admissible, and will usually be consistent (if the relaxed problem formulation respects triangle inequality)

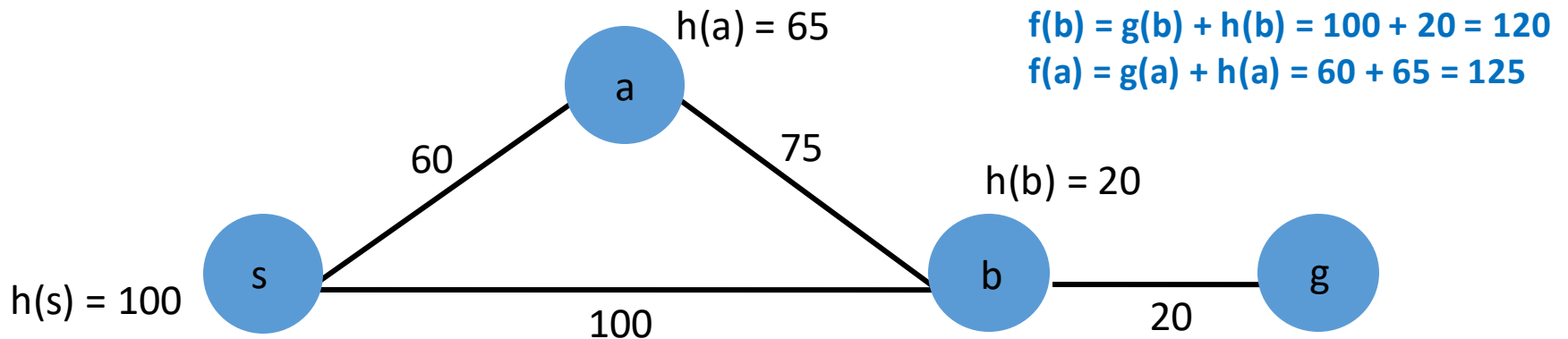
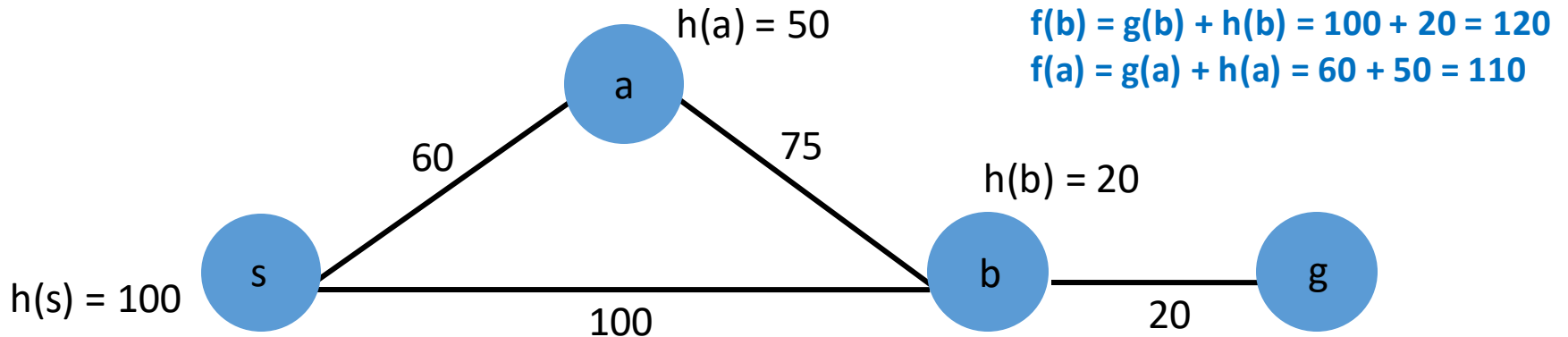
Dominance

- If $h_1(n) \leq h_2(n) \forall n$ (both heuristics are admissible)
then h_2 dominates h_1



Dominance

- Why does this matter?
 - We want as large an h as possible, or as tight a bound as possible



Example heuristic: eight-puzzle

- What would be a good heuristic for this problem?
 - $h_1(n)$ = number of misplaced tiles = 7
 - $h_2(n)$ = manhattan distance = $4+2+2+2+2+3+3=18$

5	4	
6	1	8
7	3	2

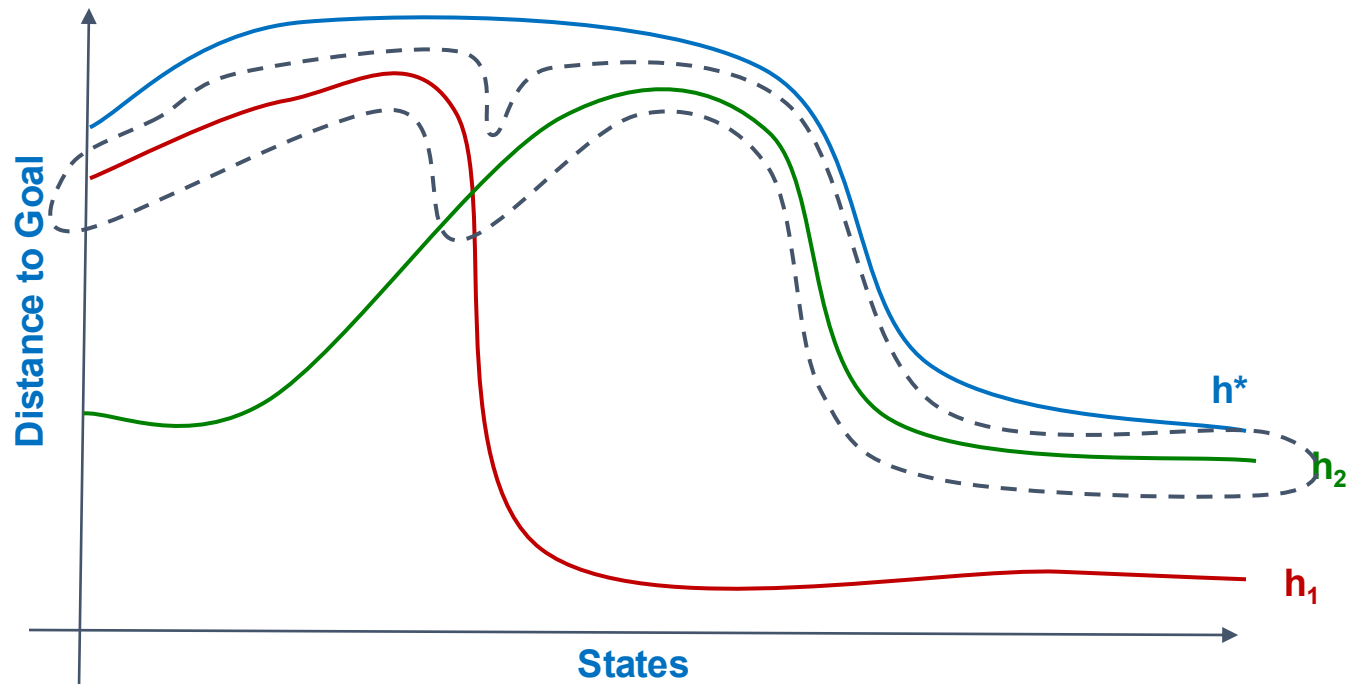
Start State

1	2	3
8		4
7	6	5

Goal State

Combining Heuristics

- If $h_3(n) = \max(h_1(n), h_2(n))$ dominates both h_1 and h_2



Properties of A*

- Complete!
- Optimal!
- Exponential worst-case time/space complexity.
 - But with a perfect heuristic, complexity is $O(bd)$, because we only expand nodes on optimal path.
 - With a good heuristic, complexity is often sub-exponential.
- **Optimally efficient:** with a given h , no other optimal search algorithm is guaranteed to expand fewer nodes than A*.
 - Technicality: up to tie-breaking

Iterative Deepening A* (IDA*)

- Depth-first search to avoid memory concerns.
- But use an f -value limit, rather than a depth limit.
- IDA* has the same properties as A* but uses less memory.
- In order to avoid always expanding new nodes, old ones can be remembered if memory permits (this version is known as Simplified Memory-bounded A*, SMA*)

Learning Heuristic Functions

- Can we automatically construct a better heuristic function for eight-puzzles?
- **Idea:** use previous experience of playing other eight-puzzles to help us learn h .
- **Need:**
 - Features that describe a particular state configuration
$$x_1(n), x_2(n), \dots$$
 - Form of heuristic function to be learned
$$h(n) = c_1x_1(n) + c_2x_2(n)$$
 - Target for the heuristic function

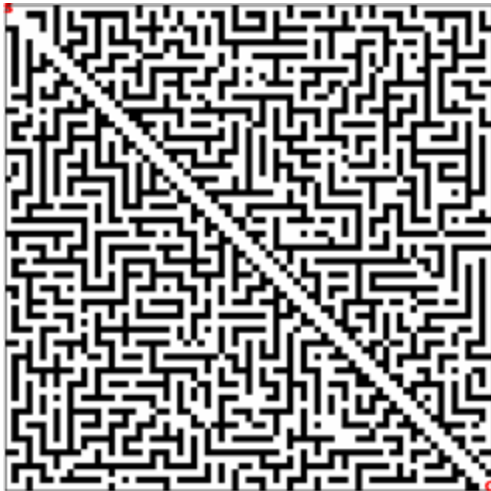
This is enrichment material, not testable. However, it can easily be a component of the project submission, and help in real-world AI projects!

Recap

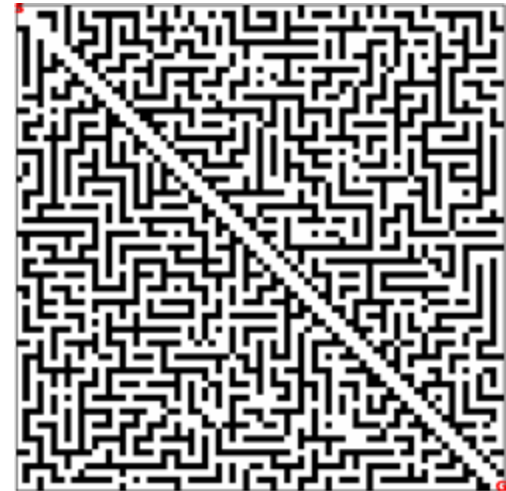
- Iterative Deepening Search (IDS)
- Uninformed Search vs Informed Search
 - Heuristics
- Best-First Search and A* Search
- Properties of Heuristics
 - Admissibility and Consistency
 - Dominance
- Iterative Deepening A*

The Goal Today

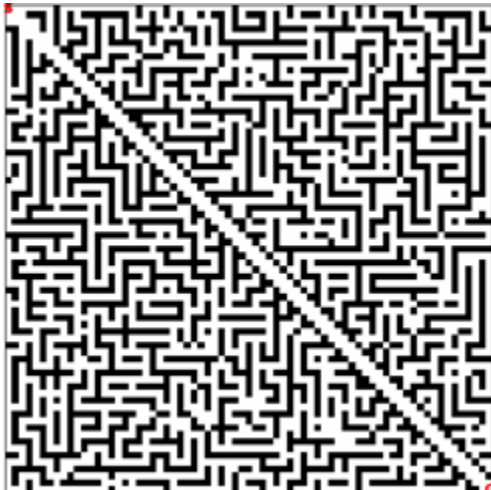
BFS



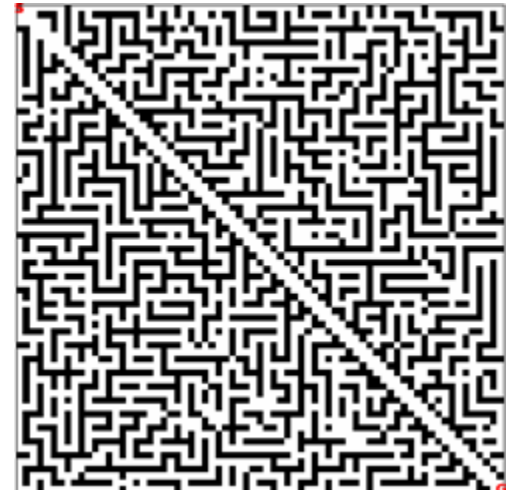
DFS



IDS



A*



Next Class

- Search for Optimization